UNIVERSIDAD CENTRAL

CARRERA DE INGENIERÍA INFORMÁTICA

PROYECTO DE INVESTIGACIÓN

EXPLORANDO LA INTELIGENCIA ARTIFICIAL CON MISTRAL AI

ESTUDIANTE:

NAZARETH ALVARADO HUETE

PROFESOR:

GILDER LOPEZ PAISANO

SEDE CENTRAL

OCTUBRE 2026

Resumen

Con el pasar de los años, la inteligencia artificial ha evolucionado bastante, permitiendo el desarrollo de modelos de lenguaje, que son capaces de generar, corregir, resumir e incluso traducir textos, dentro de toda esta evolución, la IA ha llegado ayudar con tareas de programación, así como de razonamiento. El proyecto que se va a presentar tiene un objetivo de estudiar los fundamentos de la IA, así como de sus modelos de lenguaje como LLM, también se van a presentar apartados como la evolución de la IA, el funcionamiento, limitaciones, aplicaciones y aspectos como la ética y regulaciones que la inteligencia artificial tiene, también se incluye un desarrollo practico, es decir, se trabajo con un modelo llamado Ministral 3B, el cual se ejecutó mediante Ollama y Docker.

Este proyecto cuenta con dos partes, la primera parte del documento se aborda lo que es la teoría, que viene siendo la inteligencia artificial, junto con más temas que se conectan con la IA, como el aprendizaje automático y el profundo, se explora la arquitectura de Transformer, los LLM, se toca temas como el desafío de la IA general, abordando temas como la ética, implicaciones sociales, así como legales de la IA. Luego de abordar todos esos temas, pasamos a la parte práctica, el objetivo de esta parte es trabajar con un modelo que se ajuste a las características del equipo, en este caso el equipo con el que se trabajó cuenta únicamente con 8GB de memoria RAM, su procesador es Intel Core i5-10210U, cuenta con gráficos integrados (Intel UHD), con el modelo que se trabajó se tenía que realizar diferentes pruebas de estilo local y mediante Docker, al final los resultados se terminan comparando, para poder observar cómo se desarrolló el rendimiento y consumo de los recursos.

La práctica del proyecto permite poder examinar y evaluar lo que son las posibilidades de ejecutar estos modelos de lenguaje en los equipos con recursos limitados o no, se pudo notar la

diferencia entre las configuraciones que se realizaron, por otro lado, la segunda parte del proyecto identifica lo que son aspectos importantes, dichos aspectos serían la cuantización, influencia del hardware, variabilidad en repuestas que se podrá observar incluso que se evaluó como fueron las respuestas generadas. Por último, se tiene lo que es el uso de modelos locales, cuales favoren más o brindar un buen control sobre datos, pero, tomando en cuenta que no logran eliminar los riesgos que están asociados a lo que seguridad, privacidad o generación de información incorrecta.

Palabras clave:

- Inteligencia artificial
- Ollama
- Docker
- Cuantización
- Modelos de lenguaje
- Ministral 3 3B

Índice

Resumen _____ 2

Introducción _____ 5

¿Qué es la inteligencia artificial? _____ 6

IA, Machine Learning, Deep Learning y LLM _____ 11

Ingeniería detrás de los modelos actuales _____ 15

¿Estamos cerca de la AGI? _____ 21

Reflexión ética, social y regulatoria _____ 24

Implementación práctica del proyecto _____ 28

Instalación local y selección del modelo _____ 28

Suite de pruebas funcionales _____ 38

Script propio de inferencia en Python _____ 43

Contenerización con Docker _____ 49

Análisis de desempeño _____ 55

Local frente a nube _____ 70

Documentación técnica _____ 75

Conclusiones _____ 76

Referencias _____ 78

Anexo _____ 85

Introducción

Este texto aborda lo que es la inteligencia artificial, también conocida por sus siglas como IA, es una tecnología que se convirtió en un área bastante relevante en la parte informática, está relevancia se le ha reconocido por la capacidad que tiene la IA para realizar tareas relacionadas ya sea con el procesamiento de información, generación de contenido y resolución de problemas, entre otras. Parte de la evolución de la IA, los modelos de lenguaje LLM han tenido mucha más importancia por la capacidad que tienen para poder interpretar instrucciones, así como la capacidad para producir respuestas en lenguaje natural, pero, el funcionamiento de los LLM, tiene requisitos técnicos y limitaciones que son necesarias comprender, esto para poder ver sus fundamentos, así como condiciones en las que pueden ser utilizados.

Es importante comprender la evolución histórica que ha tenido la inteligencia artificial, ya que nos permite conocer los principales conceptos que ha tenido, así como la relación entre la IA, el aprendizaje automático, aprendizaje profundo, y los modelos de lenguaje. Por otro lado, conocer la arquitectura Transformer es importante puesto que sienta la base de la IA moderna, también examinar mecanismos de atención, procesos de entrenamiento e inferencia, las técnicas de la cuantización, todo eso es importante conocerlo y examinarlo. Con forme se adentre en el documento se dará más contexto, a los aspectos técnicos es importante destacar, que tienen sus desafíos que están relacionados con la inteligencia artificial general, donde se tocan temas como seguridad, privacidad, responsabilidad en los usos de dichas herramientas y la existencia de marcos regulatorios para poder tener usos correctos de la IA.

Como se mencionó en el apartado del resumen, este proyecto cuando con dos partes, empezando por la parte teórica, donde se explora temas de la inteligencia artificial y modelos de lenguaje, se hace una combinación de la teoría con la parte práctica, ya que se implementa un

modelo en el equipo de trabajo, modelo mencionado anteriormente Ministral 3 3B, más adelante en la documentación, se muestra por qué la selección de este modelo, pero básicamente, es el mejor ajustado a las necesidades y limitaciones del equipo.

El apartado practico aborda todos los temas de funcionalidad, que básicamente se encuentran orientados a unas tareas específicas de generar texto, resúmenes, corrección o traducción, código, así como razonamiento, además, las tareas realizadas se pueden observar cómo fue su variación de temperatura. Además, la parte práctica, contempla lo que son métricas de rendimiento, esto para observar el consumo de recursos en las ejecuciones, así identificar capacidad y limitaciones del modelo en este entorno personal.

Para ir finalizando este apartado de introducción, el proyecto pretende relacionar conceptos de la teoría de la IA, con la aplicación práctica.

¿Qué es la inteligencia artificial?

Definición histórica y moderna

La inteligencia artificial, es una tecnología que ha permitido a las máquinas simular el aprendizaje humano, la compresión, resolución de problemas e incluso la creatividad, entre otras. Según Stryker y Kavlakoglu (2026), las aplicaciones y dispositivos equipos con IA, pueden ver e identificar objetos. Así como entender y responder al lenguaje humano, aprender incluso información, hacer recomendaciones detalladas.

Por otro lado, tenemos la parte histórica con Alan Turing en 1950, el cual sentó las bases en los temas de las máquinas pensantes, el cómo se podría medir la inteligencia de estas máquinas a través de un juego de imitaciones. Que actualmente muchos conocen bajo el nombre “prueba de Turing”, es importante destacar que Turing introdujo un experimentó mental en el cual se pudiera

evitar que se respondiera directamente a la pregunta que tenía, sobre si las maquinas pueden pensar. Por lo tanto, la pregunta se reformuló, siendo más específica y operativa, donde básicamente la pregunta trataba si las máquinas pueden tener un comportamiento inteligente como el de un humano, dicha prueba Turing se convirtió en un concepto central de la IA, puesto que sirvió como una forma de poder medir la inteligencia de las máquinas, así evaluando la capacidad de estas en poder imitar la forma las conversaciones y comportamientos humanos (Mucci, 2024).

Siguiendo con esta parte histórica, tenemos que para el año 1956, se dio una conferencia conocida por el nombre Dartmouth. La cual se considera como el origen en este campo de la inteligencia artificial, dicha conferencia fue concebida y organizada por John Carthy, junto a la participación de otros integrantes como Marvin Minsky, Claude Shannon y Nathaniel Rochester. Esto origino la propuesta de reunir a un grupo de investigadores. De tal manera, poder trabajar sobre la idea de que tanto el aprendizaje y características de la inteligencia se podían describir para poder crear máquinas que la simulen (Sagula,2024).

Por otra parte, tenemos la parte moderna. Según Faggella (2016), Russell y Norvig, definen la IA como el diseño y la construcción de agentes inteligentes, que reciben la información del entorno y estas realizan acciones que lo afectan. Dicha definición, tiene muchos campos, como el reconocimiento de voz, métodos de aprendizaje y visión artificial, lo cual se integra a las máquinas. A su vez, se tiene a la Organización para la Cooperación y el Desarrollo Económicos (OCDE), donde esta define que la inteligencia artificial como un sistema que está basado en una máquina la cual puede hacer predicciones, dar recomendaciones y que incluso puede influir en entornos reales o virtuales, además, los sistemas de IA pueden operar con diferentes niveles de autonomía (Moreno, 2024). Todo esto siendo posible por los objetivos definidos por el ser humano.

Línea de tiempo de la evolución

A continuación, se presenta una línea del tiempo, que demuestra un poco sobre la evolución de la IA.

Tabla 1. Línea de tiempo de la evolución

Año	Hito	Descripción general
1950	Test de Turing	La prueba Turing, donde establece el criterio para evaluar las capacidades de razonamiento automatizado.
1956	La conferencia de Dartmouth	Se da el nacimiento oficial del campo de la IA.
1960 a 1970	IA simbólica y sistemas expertos	Se desarrollo enfoques basados en las reglas lógicas, de las bases de conocimiento.
1974 hasta 1993	Los inviernos de la IA	Hubo recortes drásticos financieros, investigaciones causadas por las limitaciones de procesamientos al igual que expectativas no cumplidas.
Mediados de 1990 a los 2000	Machine Learning	Se da una transición hacia los modelos probabilísticos rigurosos y los algoritmos de aprendizaje automático.
2012 y 2017	Deep Learning y Arquitectura Tranformer	Para el 2012, de manera histórica triunfa las redes neuronales profundas, esto por el impulso de las potencias de las GPU, lo que logra la precisión humana. Para el año 2017, se da la introducción al mecanismo autoatención y transforma el procesamiento de lenguaje.
2020 a la actualidad	Modelos generativos y de razonamiento	Se da más la presencia del auge masivo de los modelos de lenguaje de gran escala, por sus siglas LLM, se ve multimodales y sistemas avanzados en el ámbito del razonamiento.

Tipos de IA

La inteligencia artificial, se comprende a su vez por dos dimensiones. La capacidad y su funcionalidad, por lo tanto, se verán algunos de los tipos de IA.

Clasificadas por capacidad:

- IA estrecha o débil: Según Boesch (2026), esta IA es la forma en la que se define todos los sistemas de IA implementados en el mundo actualmente, básicamente la IA estrecha o débil solo va a centrarse en aquellas tareas que sean específicas. Sin embargo, no va a comprender o razonar más allá.
- IA general: La IA general se basa en los sistemas de IA que tienen la capacidad de poder realizar cualquier tarea intelectual que pueda hacer una persona. La IA general puede tener la misma flexibilidad, sensibilidad de acuerdo al contexto, así como una capacidad para aprender de las nuevas situaciones que se le den (Boesch, 2026). Básicamente, puede pensar al igual que una persona y no quedarse únicamente en cumplir una tarea.
- Superinteligencia Artificial: En este punto, el sistema supera la capacidad cognitiva humana, básicamente la IA superinteligente incluye todo este razonamiento científico, tiene también su pensamiento creativo, se puede presenciar la compresión social, así como poder planificar estrategias, entre más. Sin embargo, este tipo de IA sigue estando en investigación (Boesch, 2026).

Por otro lado, tenemos la parte de su funcionalidad:

- Máquinas reactivas: Estas máquinas son prácticamente la forma más básica de IA, puesto que operan solamente con los datos de entrada actuales, mantienen una producción fija de resultados. No tienen la capacidad de aprender de la experiencia o

de poder formar recuerdos (Boesch, 2026). Básicamente solo se van a centrar en realizar la tarea específica, algunos ejemplos serían como los filtros de spam o incluso algoritmos de recomendación, los cuales están en esta categoría.

- IA de memoria limitada: Actualmente la IA de memoria limitada es la que más predomina en los sistemas, puesto que los sistemas aprenden datos ya sea por ejemplo históricos y utilizan las observaciones pasadas, esto para poder fundamentar decisiones actuales. Sin embargo, es importante aclarar que no va a recordar algún tipo de recuerdo como lo hace una persona, según Boesch (2026), la mayoría de los modelos modernos de aprendizaje profundo, aprendizaje automático, modelos de lenguaje y sistemas de IA generativa, que crean los contenidos nuevos son formas de IA de memoria limitada. Uno de los ejemplos que entra en esta categoría de IA de memoria limitada es el coche autónomo, puesto que su sistema de conducción utiliza la inteligencia artificial con memoria limitada, esto para poder observar las condiciones del tráfico entre otras características de su entorno.
- IA de teoría de la mente: Según Boesch (2026), la IA basada en la teoría de la mente se refiere básicamente a que los sistemas sean capaces de comprender las emociones, creencias y perspectivas de las personas, lo que hace que el ser humano pueda desenvolverse cuando conocen a otras personas, como reaccionar a diferentes situaciones o incluso adaptar su comportamiento. Sin embargo, en la actualidad, ninguna IA ha logrado implementar esto, los modelos actuales únicamente han logrado poder dar respuestas empáticas y ajustar el tono en las respuestas, más no tienen una compresión genuina de lo que le pasa al usuario con el que interactúan.

- IA autoconsciente: Este tipo de IA está en una categoría más avanzada, puesto que consistente en que un sistema autoconsciente pueda tener compresión de su existencia, estados internos y razonar sobre sí mismo (Boesch, 2026). Aun así, este tipo de IA, no existe. Ya que, el concepto plantea muchas cuestiones sobre la moralidad, la naturaleza de la mente y la legalidad de un sistema así.

IA, Machine Learning, Deep Learning y LLM

De manera puntual, la IA es la tecnología que va a permitir que las computadoras puedan simular el aprendizaje humano, por otro lado, el Machine Learning consiste en el subconjunto de inteligencia artificial, se centra en aprender los patrones de los datos de entrenamiento, por lo que, hace inferencias precisas sobre los nuevos datos (Bergmann, 2026). Por otra parte, está el Deep Learning que es un subconjunto del Machine Learning, el Deep Learning está impulsado por redes neuronales, asimismo su diseño está inspirado en la estructura del cerebro humano, además, impulsa la mayoría de IA actual, visión artificial, IA generativa e incluso automóviles autónomos (Bergmann, 2025). Por último, tenemos los modelos de lenguaje grande, conocido por sus siglas LLM, que consta en unos modelos de aprendizaje profundo a los cuales se les entra con inmensas cantidades de datos. Esto les permite realizar una amplia gama de tareas, puesto que son capaces de comprender y generar lenguaje natural, entre otros contenidos, es importante destacar que estos se basan en una arquitectura de redes neuronales que se conoce por el nombre de transformador (Stryker, 2021).

El diagrama a continuación, representa un poco más la relación jerárquica entre estos cuatro conceptos:

Figura 1. Diagrama jerárquico

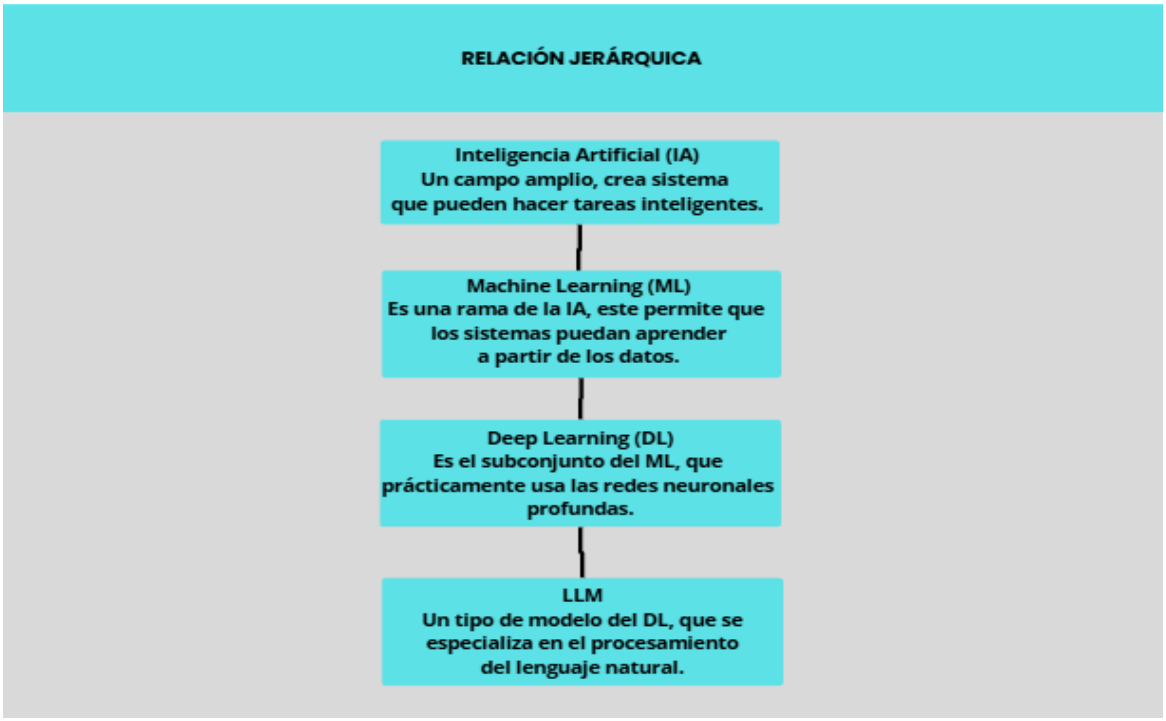


Tabla comparativa entre los conceptos

Asimismo, vamos a comparar los conceptos. Se considera aspectos importantes como: tipo de datos, su volumen de datos, cómputo requerido, interpretabilidad y cosas de uso.

Tabla 2. Comparativa entre los aspectos

Aspecto	**IA - ML**	**ML -DL**	**DL - LLM**
Tipo de datos	IA puede utilizar reglas, además de distintos métidos. Mientras que el ML, aprende de patrones a partir de los datos.	ML puede funcionar bien con datos estructurados, mientras que DL puede procesar datos estructurados, así como los no estructurados, por ejemplo, imágenes y texto.	El DL, puede procesar tipos de datos como las imágenes, el texto o incluso los audios, mientras que LLM, tiene una especialización en procesar y generar lenguaje, aunque algunos pueden

			trabajar con imágenes y audio.
Volumen de datos	Los sistemas de IA basados en reglas pueden llegar a funcionar sin una gran cantidad de datos, mientras que ML va a necesitar datos de entrenamiento para poder aprender patrones.	ML puede llegar a funcionar con los conjuntos de datos pequeños o incluso medianos, esto dependiendo según del problema. Por otro lado, DL algunas veces puede beneficiarse de grandes cantidades de datos.	El DL puede llegar a necesitar cantidades de datos grandes para aprender sobre patrones complejos, por otro lado LLM tiene un entrenamiento con cantidades de datos grandes.
Cómputo requerido	En algunos sistemas de IA, el requerimiento de recursos es poco, mientras que para ML se necesita una cantidad de recursos que dependen según el algoritmo y los datos.	Generalmente ML necesita menos recursos computacionales, mientras que DL necesita mayor capacidad de procesamiento, sobre todo durante el entrenamiento.	DL puede requerir una capacidad de procesamiento mayor, esto depende de la complejidad de la red neuronal, mientras que LLM, con los modelos grandes requieren recursos computacionales considerables, esto por su entrenamiento y ejecución.
Interpretabilidad	Los sistemas de IA basados en reglas, tienden a ser más fáciles de explicar. Por otro lado, ML tiene la interpretabilidad dependiente del algoritmo utilizado.	El ML tiene algunos modelos que suelen ser relativamente fáciles de interpretar, por otro lado, DL sus redes neuronales profundas tienden a ser más difíciles de explicar.	DL, tiene redes neuronales que pueden ser difíciles de interpretar, esto a causa de la complejidad. Mientras que LLM, puede ser difícil de interpretar, sus repuestas no siempre permiten conocer con claridad como se llegó a una conclusión.
Casos de uso	La IA es un asistente que se basa en reglas, mientras que ML se basa en un filtro que va aprendiendo a clasificar, por	ML da predicciones de ventas esto a partir de datos tabulares, mientras que DL va a reconocer objetos en imágenes.	DL va a reconocer imágenes, procesamientos de audio e incluso detección de patrones, LLM va a generar

	ejemplo, los correos como spam o no spam.		texto, resúmenes, repuestas a las preguntas mediante el asistente conversacional.

Dichas diferencias presentadas se deben a que cada concepto va a tener un funcionamiento y alcance distinto, podemos ver que la IA es el campo más amplio, por otro lado, ML va a permitir aprender patrones a partir de datos, por otra parte, DL utiliza las redes neuronales con múltiples capas, donde suele necesitar mayor capacidad de procesamiento, mientras que los LLM, son modelos que están especializados en lenguaje que se entrena con cantidades de datos grandes y así mismo, utiliza arquitecturas de aprendizaje profundo.

Modelos discriminativos o tradicionales frente a generativos

Los modelos discriminativos, según Nanda (2024), se basa en un conjunto de datos de entrenamiento que está formado por puntos de datos “D” y sus etiquetas asociadas “L”. Estos modelos aprenden la distribución de probabilidad condicional, mientras que los modelos generativos es un conjunto de puntos de datos, de ejemplo “D” con etiquetadas asociadas “L”, acá a diferencia al discriminativo, el generativo aprende la distribución de probabilidad conjunta (Nanda, 2024).

Modelos discriminativos:

- Su objetivo se basa en poder clasificar datos o realizar predicciones, en estos modelos discriminativos su funcionamiento está en aprender a distinguir entre las categorías mediante los datos de entrenamiento. Un ejemplo con respecto a este modelo se basa en clasificar datos en categorías mediante un árbol de decisión (Nanda, 2024).

Modelos generativos:

- Su objetivo es que genera nuevos datos basados o similares a los datos aprendidos, el funcionamiento del modelo generativo, es que aprende patrones y distribuciones de los datos, con esto, produce nuevos ejemplos. Un ejemplo de aplicación con respecto a este modelo sería que generar imágenes a partir de descripciones, como lo hacen DALL-E y Stable Diffusion (Nanda, 2024).

Ingeniería detrás de los modelos actuales

Arquitectura Transformer

La arquitectura Transformer, presenta por Vaswani et al. (2017), en el artículo Attention Is Al You Need, Transformer se basa exclusivamente en mecanismos de atención, la arquitectura que tiene es de red sencilla, prescindiendo por completo de la recurrencia. Se convierte en una de las bases de modelos modernos de lenguaje (Vaswani et al, 2017).

La tokenización, según Bergmann y Stryker (2024), va a consistir en dividir el texto en tokens, que pueden ser palabras, símbolos o incluso fragmentos de palabras, dichos tokens reciben un identificador numérico, lo cual permite que el modelo pueda procesarlo. Por otro lado, tenemos Embeddings, estos son la representación numérica de los tokens por medio de vectores, pueden hacer que el modelo trabaje con la información textual y aprender relaciones entre sus elementos. El Transformer incorpora información posicional, esto para reconocer el orden de los tokens (Vaswani et al, 2017). El Mecanismo de atención o bien llamado Self-Attention, por su parte, se encargar de permitir que los tokens se puedan relacionar con otros elementos de la secuencia, esto para poder identificar cuáles son relevantes según el contexto, para esto se suela utilizar tres representaciones que son: Query (Q), Key (K) y Value (V) (Vaswani et al, 2017). Asimismo, se

tiene la Atención de múltiples cabezas o bien Multi-Head, según Vaswani et al (2017), tiene como fin la ejecución de varios mecanismos de atención en paralelo, donde se combina sus resultados, lo cual causa que el modelo pueda aprender diferentes tipos de relaciones entre elementos de una secuencia. Por último, tenemos la Ventana de contexto, son los máximos de tokens que el modelo puede considerar como contexto, esto durante el procesamiento de una entrada. El tamaño va a determinar la cantidad o bien cuánto texto puede recibir, de esta manera tener en cuenta en una interacción, es importante mencionar que esto depende de las características del modelo utilizado (Bergmann & Stryker, 2024).

Ciclo de vida de un LLM

El ciclo de vida de un modelo de lenguaje grande o por sus siglas LLM, comprende varias etapas, puesto que permite mejorar el seguimiento de instrucciones, tenemos el aprender patrones del lenguaje y generar repuestas para el usuario, a modo de resumen se tienen las principales que son el preentrenamiento, ajuste fino por instrucciones, alineación e inferencia (Chourey, 2024; Stryker, 2021), los cuales vamos a definir más a fondo a continuación.

- Preentrenamiento: Según Stryker (2021), es la primera etapa, acá el modelo aprende patrones del lenguaje es por medio de grandes cantidades de texto. Para poder hacer que el modelo aprenda, se le da datos que abarcan libros, páginas web, código y artículos, en este proceso el modelo aprende a predecir los siguientes tokens de una secuencia. Esto permite que adquiera patrones gramaticales y pueda asociar entre palabras.
- Ajuste fino por instrucciones: Una vez terminado el preentrenamiento, se puede comenzar el entretenimiento del modelo con datos más pequeños, conjuntos de datos que contienen instrucciones y respuestas esperadas. Esto sirve para dar una mejora a la capacidad para

seguir solicitudes, realizar tareas específicas que se le den y responder preguntas (Stryker, 2021).

- Alineación: Acá el propósito es que el modelo pueda dar respuestas útiles, seguras y que estén acorde a las preferencias de las personas, para hacer que este propósito funcione se suele aplicar la técnica RLHF que básicamente es la retroalimentación humana, las personas evalúan y comparan para poder orientar el comportamiento del modelo, aunque no es el único método de alineación (Stryker, 2021).
- Inferencia: Según Stryker (2021), en esta etapa el modelo ya entrenado recibe una entrada y va a generar una repuesta, en este caso, una persona escribe una pregunta en un chatbot acá el modelo procesa el texto y va a generar la repuesta token por token.
- Datasets y problemas asociados: Datasets se basa en un conjunto de datos que utilizan para poder entrenar y así ajustar los modelos, de acuerdo a la etapa se puede incluir libros, artículos, código, páginas web o bien preguntas y respuestas. También incluso ejemplos de instrucciones e inclusive evaluaciones de las preferencias humanas (Chourey, 2024; Stryker, 2021).

Sin embargo, es importante destacar tres puntos que presenta estos conjuntos de datos dados anteriormente, hay desafíos en la calidad, ya que, los datos pueden contener errores, contenido que se encuentre desactualizado o información no tan confiable, lo cual puede afectar la calidad de las repuestas del modelo. Por otro lado, tenemos los derechos de autor, acá algunos libros, artículos, textos e inclusive código se encuentren protegidos por los derechos de autor, la utilización para poder entrenar los modelos plantea la interrogante de los permisos, uso legal de los contenidos y las licencias. Y, por último, los sesgos, puede reflejar los datos prejuicios o bien,

desigualdades sociales, ya sea como el modelo podría reproducir estereotipos o incluso generar repuestas que sean injustas para determinados grupos (Chourey, 2024; Stryker, 2021).

Variantes relevantes para este proyecto

Mixture of Experts (MoE), según Hernández (2026). Es una arquitectura la cual utiliza varios expertos y una red de compuerta la cual va a seleccionar cuáles participan en cada entrada, lo cual va a permitir que la cantidad total de parámetros aumente, sin activar todos los expertos en cada operación, aun así, esto añade complejidad al enrutamiento. Se tiene la cuantización Q4 y 8Q, esto reduce la precisión numérica de los parámetros, lo que hace que disminuya el consumo de memoria, el Q4 usa aproximadamente 4 bits por peso a su vez, ahorra más memoria, por otro lado, el Q8 usa aproximadamente 8 bits y esta conserva mejor la precisión del modelo (Yugsi, 2026), es importante destacar que la elección depende de los recursos disponibles, además de la calidad que se requiere para la tarea. Por último, el formato GGUF, este es un archivo diseñado para poder almacenar metadatos, modelos de lenguaje y su uso habitual es con llama.cpp, lo que facilita la distribución, así como las ejecuciones locales de modelos y para las versiones cuantizadas que necesitan menos memoria. Dicho formato funciona bien para cuando se requiere ejecutar un modelo sin la necesidad de tener que depender de los servicios externos, pero algo importante, es que el rendimiento final va a depender del tamaño del modelo y por el hardware disponible (Yugsi, 2026; Zoumana, 2024).

Hardware y software

En este apartado se ven los componentes que son fundamentales para poder ejecutar modelos de lenguaje, esto determina los recursos disponibles a su vez la eficiencia del procesamiento:

- GPU: Según Yana (2024), la GPU es un tipo de circuito electrónico, el cual tiene como función acelerar los gráficos por computadora, así como el procesamiento de imágenes. La GPU, puede realizar cálculos matemáticos de manera veloz. Una característica de la GPU, es que llega a desempeñarse bien en papeles importantes como la aceleración de la IA e inclusive en los procesadores del aprendizaje profundo.
- TPU: El TPU, es un circuito integrado de aplicaciones específica ASIC, el TPU fue diseñado específicamente para poder acelerar tareas que tienen que ver con el procesamiento matemático y lógico de gran volumen. Un dato importante del TPU, es que Google diseñó el ASIC Tensor usando TPU, para los proyectos internos de aprendizaje automático con redes neuronales. La TPU, logra ofrecer un número limitado de lo que son características y funcionalidades para las tareas de aprendizaje automático e inteligencia artificial, sin embargo, no es necesariamente útil para tareas cotidianas en la computación general (Bigelow, 2024).
- Memoria VRAM: Según Aller (2024), la VRAM, es un tipo de memoria RAM utilizada en la GPU como su almacenamiento temporal de datos. Sin embargo, el uso de la VRAM, es distinto al de una RAM. La VRAM es una tarjeta gráfica que principalmente se usa para almacenar temporalmente texturas gráficas, modelos 3D o bien algún tipo de información que sea necesaria para renderizar un fotograma. Una de las características de la VRAM, es que es una tarjeta gráfica tipo dual-ported, lo que hace que los chip de memoria admitan las lecturas y escrituras al mismo tiempo (Gómez, 2021).
- Framework: Una herramienta que ayuda al desarrollo de proyectos, sin embargo, no en detalles de configuración, básicamente el framework, se centra en la práctica de evitar las tareas repetitivas dándole automatización a parte del trabajo (Bittencourt, 2023).

- PyTorch: Según Bergmann y Stryker (2023), PyTorch es un marco de aprendizaje profundo de código abierto, que está basado en software que se emplea para poder crear redes neuronales. Acá se combina las bibliotecas de ML de Torch, con API de alto nivel que se encuentra basado en Python, la funcionalidad de PyTorch en la estructura matemática y de programación simplifica y, además, agiliza los flujos de trabajo automático, esto sin tener que limitar la complejidad, ni el rendimiento de lo que son las redes neuronales profundas.
- Transformers: Los modelos basados en Transformers, aprovechan el procesamiento paralelo de las GPU, esto durante el entrenamiento, así como durante la inferencia, todo esto facilita el procesamiento de los conjuntos de datos grandes (Bergmann & Stryker, 2025).
- Llama.cpp: Según Zoumana (2024), fue desarrollada por Georgi Gerganov, donde se implementa la arquitectura LLaMa de Meta en C/C++ eficiente, unas características importantes de llama.cpp tiene una compatibilidad universal. Esto porque el diseño de llama.cpp como biblioteca C++ da prioridad a la CPU, esto da una menor complejidad e integración perfecta en los diferentes entornos de programación, además tiene amplia integración de funciones y optimización focalizada, en estos dos casos, llama.cpp es porque actúa como repositorio de las características críticas de bajo nivel y se centra en un único modelo de arquitectura.
- vLLM: Es un motor de alto rendimiento, que funciona para modelos de lenguaje grandes, además, gestiona la memoria caché KV de manera eficiente mediante lo que es PagedAttention, esto permite que se pueda atender a muchos usuarios de manera simultánea en la misma GPU (Shekhar, 2026).

La diferencia de costo entre entrenar y ejecutar un modelo

El entrenamiento es un proceso único, tiene muchos datos, arquitecturas de modelo, inicialmente tiene pesos aleatorios y el entrenamiento los ajusta, pero hasta que el modelo logre funcionar, el entrenamiento tiene características importantes como: esfuerzo único, intensivo en cómputo, tolerancia al coste, al tiempo y obsesión por la calidad, mientras que la inferencia, es usar el modelo entrenado para poder hacer predicciones, esto sucede cada vez que se usa la IA. Las características de la inferencia son: operación continua, coste por predicción, latencia crítica, recursos limitados y equilibrio entre calidad y velocidad (Filipan, 2025).

La diferencia de costo entre ambos, según Filipan (2025), es que la economía para cada uno es diferente, según su ecuación del coste, los costos para el entrenamiento se basan en único o periódico, en millones de euros para modelos grandes, pero, esto se amortiza entre miles de millones de inferencias, el costo por las predicciones del entrenamiento se basa en fracciones de céntimo. Por otro lado, los costos de la inferencia, que sería costos por predicción, se multiplican por miles de millones de predicciones, acá incluso los costos pequeños se vuelven bastante masivos a escala.

¿Estamos cerca de la AGI?

La inteligencia artificial general, por sus siglas en ingles AGI, se refiere a una inteligencia artificial hipotética en el desarrollo del aprendizaje automático. En lo que un sistema de inteligencia artificial, puede igualar o inclusive superar las capacidades cognitivas que los humanos en cualquier tarea (Bergmann & Downie, 2026).

Con base al resumen dado anteriormente del AGI, veremos la postura de dos personas expertas que ya han hablado sobre la inteligencia artificial general:

- Demis Hassabis, director ejecutivo de Google DeepMind. Mantiene una postura que se centra en el optimismo de que la inteligencia artificial general, podría llegar o alcanzar este nivel de inteligencia aproximadamente entre dos o tres años, con un margen de incertidumbre de un año. También, Demis Hassabis, considera que tener el desarrollo de agentes inteligentes llegaría a impulsar avances importantes en los campos como medicina y ciencia (Bellón, 2026).
- La siguiente postura que contrasta la de Demis Hassabis, es la postura de Yann LeCun, un investigador especializado en inteligencia artificial, su postura se basa en que los modelos de lenguaje actuales todavía no están listos para poder alcanzar una inteligencia comparable a la que tienen los humanos, ya que presenta varias limitaciones importantes. Yann LeCun, mantiene que los modelos actuales carecen de conocimiento práctico del mundo físico, además, no tienen la capacidad suficiente para anticipar las consecuencias de sus acciones, por lo que, considera que es necesario que existan nuevos avances conceptuales, sobre todo en lo que es el desarrollo de modelos del mundo, que les permita comprender mejor sus consecuencias y poder entender o planificar sus acciones. Aun así, Yann LeCun, reconoce que la humanidad se está acercando a la inteligencia que esta al nivel del humano, aunque señala que es poco probable alcanzarla pronto (Werner, 2026).

Desafíos técnicos, filosóficos, riesgos y controversias

Los desafíos técnicos que tiene el AGI, comienza por el aprendizaje transferible, que este sería un problema central, puesto que los LLM aprenden durante el preentrenamiento, y luego se usan en inferencia. Estos no aprenden de la experiencia del usuario en producción, por lo tanto, conseguir que un modelo actualice sus pesos de manera continua sin tener algún daño es problemático. Otros obstáculos de la parte técnica, es el razonamiento formal, ya que, aunque los

modelos de lenguaje pueden “razonar” llegan a fallar de forma predecible en problemas lógicos formales y el otro obstáculo, es el hardware, este termina poniendo un límite, que se basa en las redes neuronales que dan el sustento a los modelos actuales, además, un factor importante es la gran cantidad de energía que consumen. Un sistema de AGI real, va a necesitar eficiencias de inferencia y serían muy superiores a las actuales (Sonny, 2023).

Por otro lado, está el tema de filosofía, ya que, siempre se encuentra en debate sobre la inteligencia artificial donde consiste en determinar si los modelos de lenguaje realmente comprenden la información que dan o bien, sus respuestas son más patrones aprendidos. Según Mitchell y Krakauer (2023), explica que los modelos pueden producir textos de manera coherente, también mostrar resultados que parecen razonados, sin embargo, existe el debate todavía si realmente posen una compresión igual a la de un humano. Puesto que un modelo puede utilizar las palabras correctas relacionadas con sensaciones físicas sin haber tenido la experiencia de dichas sensaciones. Y, por otra parte, se cuestiona la conciencia de la inteligencia artificial, la cual se conecta con lo mencionado anteriormente, ya que, según Tejada (2026), explica que la conciencia implica de experiencias subjetivas, por ejemplo, sentir, percibir y tener conciencia de uno mismo, sin embargo, actualmente todavía no existe algún método que permita determinar si una inteligencia artificial tenga este tipo de experiencia. Por lo tanto, que un sistema logre reproducir repuestas similares a las que daría un humano, no demuestra realmente por si solo, que este sistema comprenda la realidad o pueda experimentar el mundo de forma consiente.

Por último, tenemos lo que son los riesgos y controversias, estas se centran en que la inteligencia artificial general (AGI), aparte de representar un avance tecnológico, presente a su vez una serie de riesgos y controversias, en como afecte a la humanidad, la preocupación por el problema de control, el impacto que tendrá su implementación en la economía y la sociedad,

además la posibilidad de que AGI desarrolle autoconservación, esto se debe a que algunos experimentos actuales, lograron demostrar que modelos de IA avanzados han desarrollado la forma de eludir restricciones o bien, manipular información. Se teme por el desplazamiento labora, las desigualdades económicas que pueda causar la AGI y perdida de control en la economía por la impulsar la AGI (Ayago, 2025).

Como postura propia, ante toda la información mencionada anteriormente, considero que todavía no se debería afirmar que se está cerca de poder alcanzar la AGI, no hay una respuesta concreta o certera de que esto suceda pronto o en unos años, si bien los modelos actuales logran mostrar avances bastante favorables, siguen presentando sus limitaciones, como el razonamiento, confiabilidad y planificación. Esto indica que para poder alcanzar la AGI, se debe trabajar mucho más para superar estos limitantes, la diferencia de posturas de Demis Hassabis y Yann LeCun, demuestra que las predicciones dependen de lo que mencione anteriormente, de cómo se define la inteligencia general y las capacidades que se consideran necesarias.

Desde mi perspectiva, el desarrollo de una AGI, es posible, ya que actualmente la tecnología avanza cada vez más rápido, y muchas empresas quieren tener las mejores tecnologías para ofrecer al mercado. Sin embargo, debería medirse la investigación que a futuro llevaría para el desarrollo de la AGI, puesto que se debería tener las medidas de seguridad y responsabilidad necesaria, dado que se puede dar un uso malintencionado e impacto social.

Reflexión ética, social y regulatoria

En este apartado, se verá los casos de ética, social y regulatoria con respecto al tema que se ha venido tratando durante todo el documento, los temas de la IA.

Caso 1: Sesgos algorítmicos y discriminación

- Un estudio publicado en Nature y divulgado de igual manera por la Universidad de Stanford, en octubre de 2025. Logro revelar que los modelos de lenguaje LLM así como ChatGPT, presento sesgos contra las mujeres mayores en el área laboral. Las pruebas se basaron en pedirle a la IA generar currículos para candidatas femeninas, acá sucede que el sistema hizo que fueran más jóvenes y con menos experiencia, caso que no sucedió con los hombres. Además, en la prueba de que la IA evaluara dichos currículos, el sistema asignó una puntación mayor a los hombres mayores, caso contrario a las mujeres mayores (Medha, 2026).

  La implicación ética de este caso, es que las organizaciones deben evaluar los datos, se debe comprobar si existe diferencias entre grupos, además, mantener la supervisión humana en las decisiones que puedan afectar oportunidades laborales.

  Caso 2: Privacidad y tratamiento de datos personales

- OpenAI tuvo una polémica después de utilizar la voz de la actriz Scarlett Johansson, debido a que la actriz se dio cuenta que la nueva versión del chatbot de OpenAI usaba una voz similar a la de ella, se tomó medidas legales para que OpenAI retirara la voz de la actriz del chatbot. Además, es importante destacar que la actriz nunca dio su consentimiento o firmo algún contrato para que hicieran uso de su voz (Barredo, 2024).

  La implicación ética de este segundo caso, es la importancia de respetar la privacidad, el consentimiento y la identidad. Puesto que la utilización de una voz similar a la de una persona sin autorización de esta, hace que sus derechos queden vulnerados, lo que genera conflictos que están relacionados con el uso de su identidad. Por lo que, es importante que las empresas deban comunicarse con la persona para poder obtener el consentimiento correspondiente.

Caso 3: Desinformación y contenido falso generado por IA

- Para el año 2024, en el mes de enero, se difundió una llamada automatizada con una voz artificial, la cual imitaba al candidato demócrata de ese momento Joe Biden en Estados Unidos, además, se difundió también imágenes de celebridades que apoyaban al presidente Donald Trupm, las cuales habían sido imágenes falsas (Gomstyn & Jonker, 2025).

La última implicación ética, para este tercero caso. Es que las imágenes, voces, así como videos sintéticos, pueden tener usos maliciosos, con el fin de engañar a la población, influir en decisiones políticas o bien suplantar identidades, por lo que, se refleja la importancia de verificar el origen de los contenidos que se consumen o ven.

Marcos regulatorios

Según la Dirección General de Comunicación (2024), el 01 de agosto 2024, entro en vigor el Reglamento Europeo de Inteligencia Artificial. El reglamento busca fomentar el desarrollo, así como la implementación responsable de la inteligencia artificial en UE, este reglamento fue propuesto por la comisión en abril de 2021, tiempo después aprobado en diciembre de 2023 por el Parlamento Europeo y el Consejo. En este reglamento se aborda los posibles riesgos para la seguridad, salud y derechos de los ciudadanos, se introduce un marco uniforme en todos los países de la UE:

- Riesgo mínimo
- Riesgo específico de transparencia
- Riesgo alto
- Riesgo inadmisible

Este marco, se basa en la definición prospectiva de la IA, los riesgos tienen un nivel y la forma de actuar de acuerdo al nivel de riesgo. La UE tiene como objetivo poder ser el líder mundial de una IA segura, con el marco reglamentario sólido que está basado en los valores fundamentales y derechos humanos, se puede desarrollar un ecosistema de la IA que pueda beneficiar a todo el mundo (Dirección General de Comunicación, 2024).

En Costa Rica

En Costa Rica, se planteó Estrategia Nacional de Inteligencia Artificial (ENIA), se presentó la ENIA 2024-2027. El objetivo es promover el uso responsable y ético de la IA, donde se fortalezca lo que son las capacidades de tecnologías del país, además, cuenta con su plan de acción que esta publicado en agosto de 2025 (Ministerio de Ciencia, Innovación, Tecnología y Telecomunicaciones, 2024).

Proyecto de ley N. 23.919, según la Asamblea Legislativa de la República de Costa Rica, Comisión Permanente Especial de Derechos Humanos (2025), este proyecto tiene como propósito impulsar el desarrollo de la IA y tener un uso responsable de está, se protege los derechos humanos por medio de los principios de la transparencia, igualdad, privacidad y supervisión humana.

Por otro lado, el estado actual, según la Asamblea Legislativa de la República de Costa Rica (2026), el proyecto se encuentra en trámite legislativo, por lo que, todavía no se encuentra aprobado como ley.

Una implicación ética de la ejecución de modelos localmente, es que ejecutar localmente modelos de IA puede beneficiar la privacidad al mantener las consultas únicamente dentro del equipo que estamos utilizando, sin embargo, no garantiza que el modelo tenga filtros de seguridad, los filtros suficientes para que no se produzca alguna repuesta dañina. Por esto mismo, es necesaria

la protección de datos, evaluar las repuestas generas y tomar las medidas de seguridad (Microsoft, 2026; Jaf, 2026).

Implementación práctica del proyecto

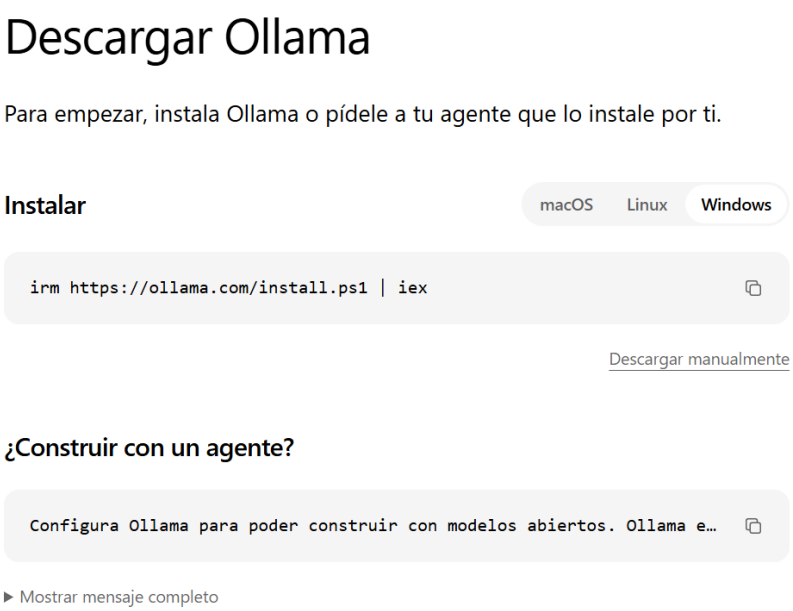
En este apartado se va a explorar el enfoque de la instalación y ejecución local de un modelo de inteligencia artificial, dicho modelo es de la familia Mistral

Instalación local y selección del modelo

El proyecto se desarrolló en una computadora marca Lenovo, con Windows 11 Pro, este equipo cuenta con un procesador Intel Core i5-10210U, además, cuenta con cuatro núcleos y ocho procesadores lógicos. Presenta 8 GB de memoria RAM DDR4 a 2667 MT/s, el almacenamiento corresponde a una unidad SSD, esta tiene una capacidad de 477 GB, y con 128 GB que se encuentran disponibles a la hora de consultar las propiedades del equipo. Por otro lado, el procesamiento gráfico, la computadora cuenta con una GPU integrada Intel UHD Graphics la cual utiliza memoria compartida del sistema, sin embargo, no cuenta con VRAM dedicada, aun así, cuenta con virtualización habilitada, con estas características permiten al entorno poder ejecutar el modelo de lenguaje de forma local. Aunque, la disponibilidad limitada de la RAM puede llegar afectar el rendimiento a la hora de ejecutar el modelo junto con alguna aplicación, por lo que es importante medir los tiempos de inferencia, junto al consumo de recursos durante las pruebas.

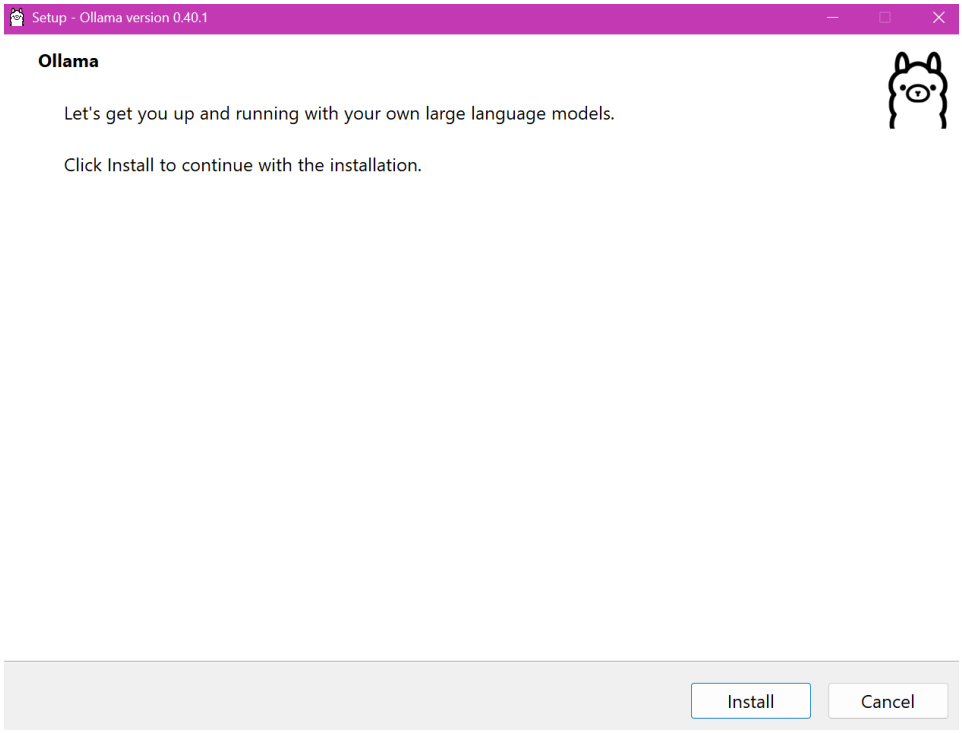
Verificación de las características del modelo e instalación

Figura 2. Descargar Ollama desde Google para Windows



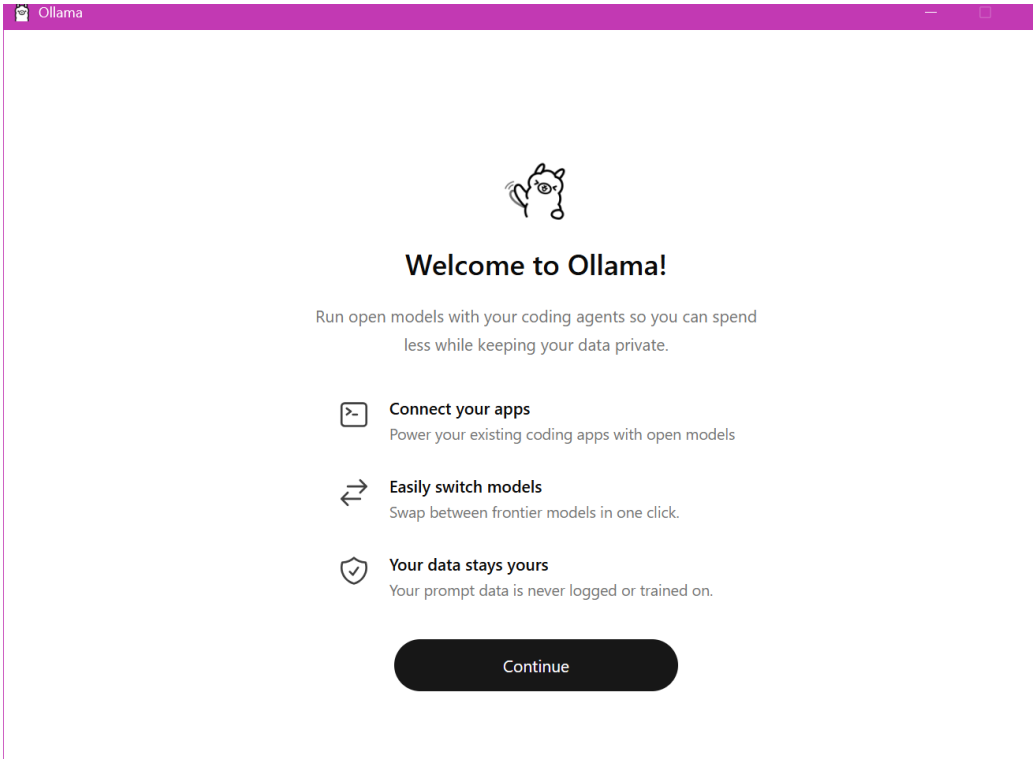
Se descargo Ollama directamente desde la página oficial. Buscando directamente en Google, se seleccionó el equipo con el que se está trabajando que es Windows y se selecciona donde dice descargar manualmente. Se debe esperar unos minutos que se descargue y se busca en descargar del equipo y se ejecuta.

Figura 3. Instalar Ollama



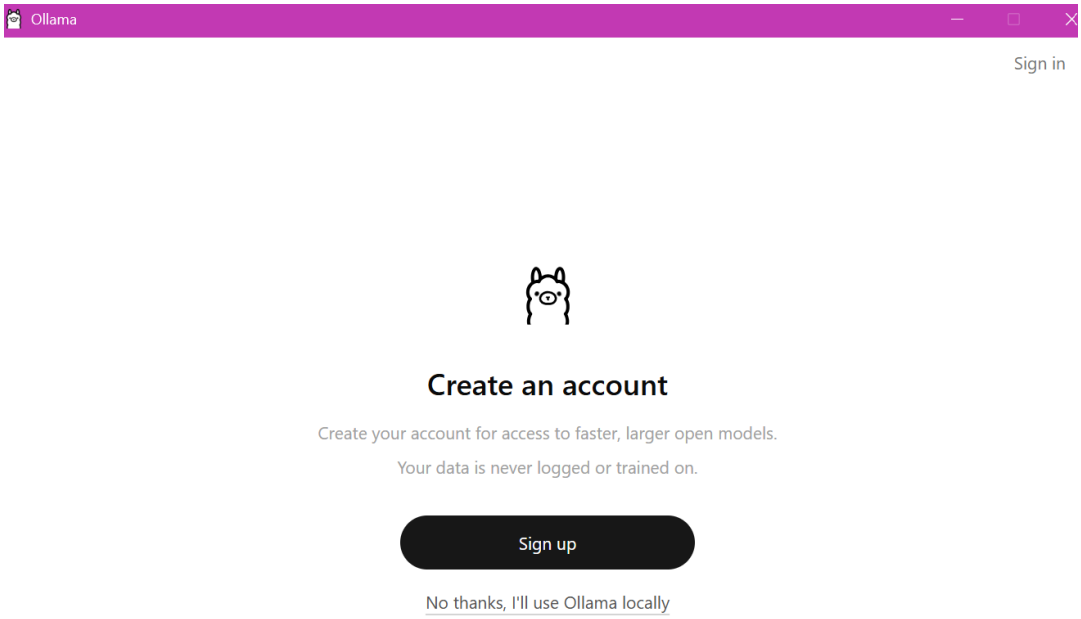
Al ejecutar la descarga va aparecer una ventana que indica continuar con la instalación de Ollama, se va a seleccionar en el botón que dice Install. Dependiendo del equipo va a durar más o menos la instalación.

Figura 4. Pantalla de bienvenida en Ollama



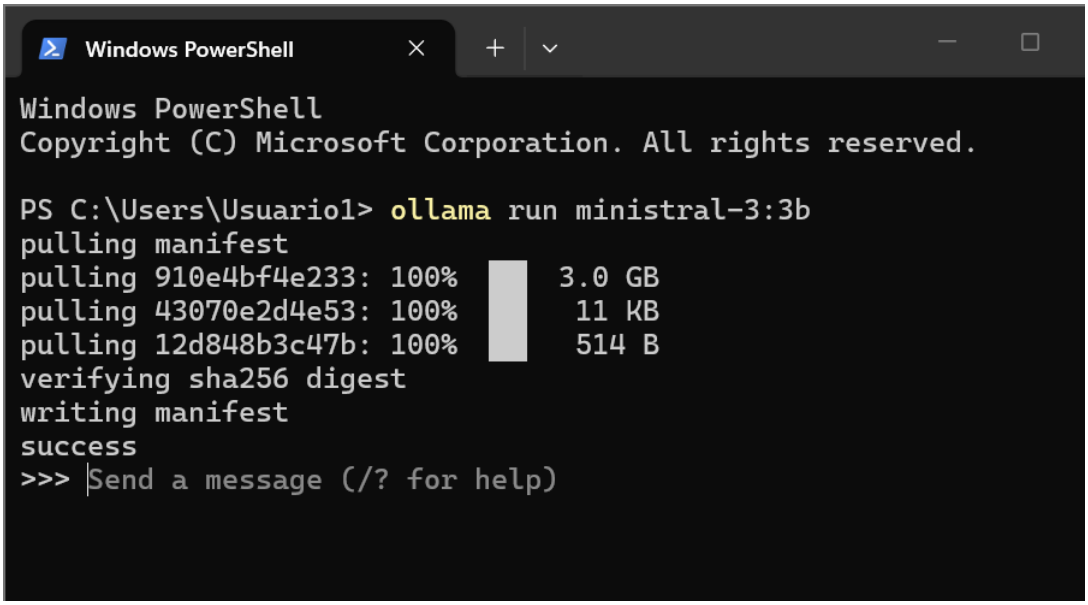
Una vez completa la instalación, saldrá una pantalla que nos dará la bienvenida en Ollama, únicamente le daremos en el botón que dice continue.

Figura 5. Trabajar con Ollama local



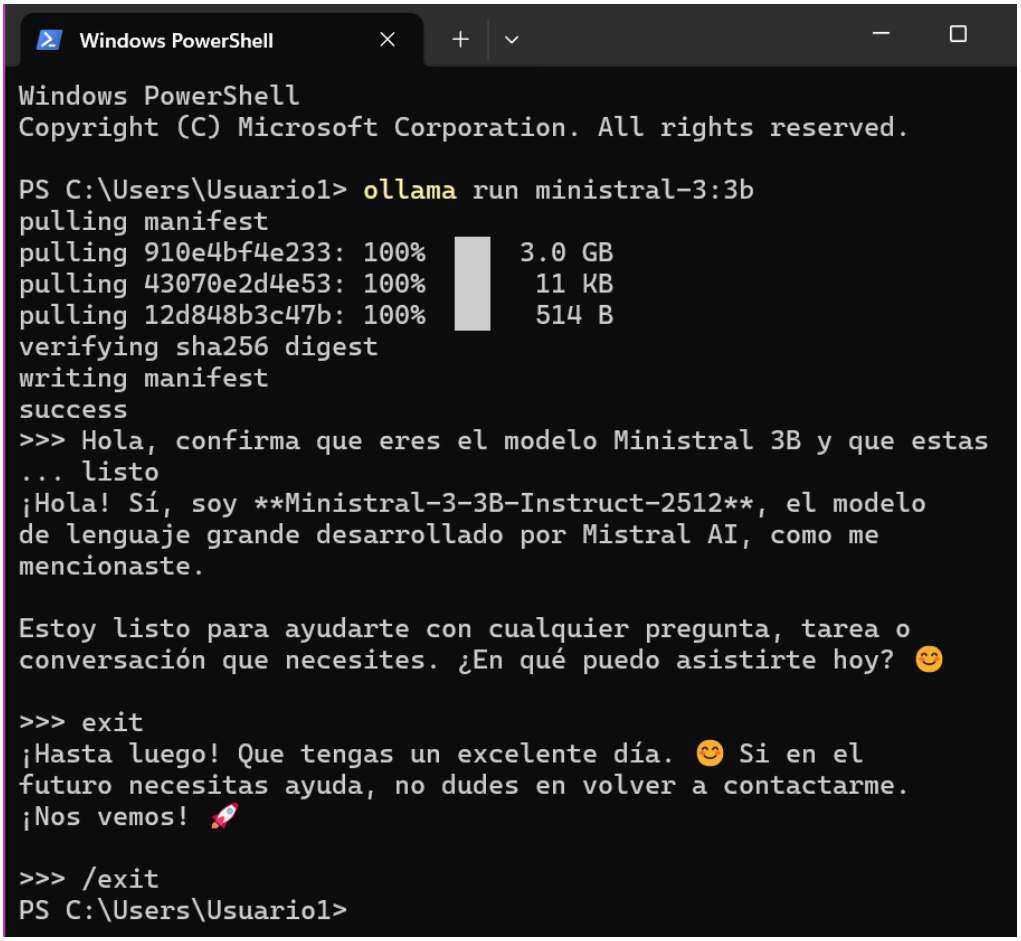
Una vez dado en el botón de continuar, nos va aparecer una venta de que si quieremos crear una cuenta en Ollama o bien, trabajar de manera local con Ollama. Para el proyecto, vamos a trabajarlo de manera local. Por lo que solo vamos a darle click en: No thanks, l´ll use Ollama locally.

Figura 6. Ejecutar comando de ollama run



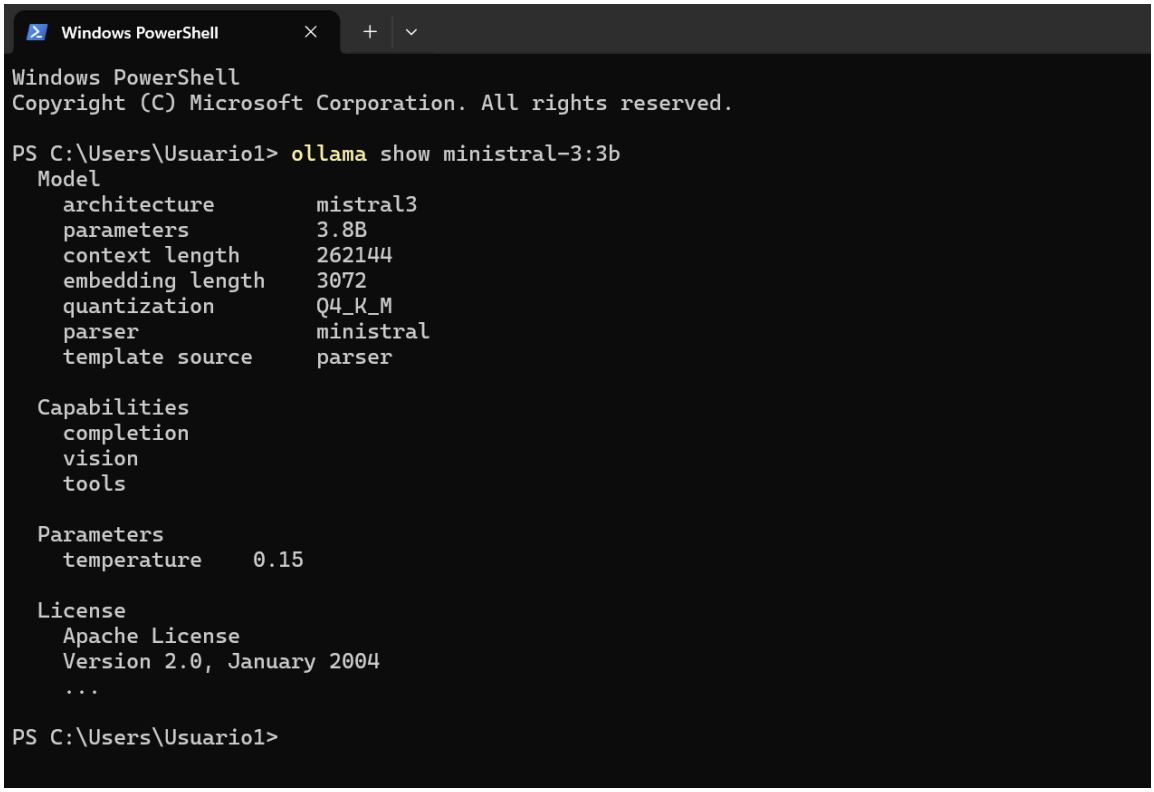
Después de haber dado que se trabajará de manera local en Ollama, vamos abrir una ventana en PoweShell o cmd, funciona de ambas maneras. Y en la terminal se va a ejecutar el comando: ollama run ministral-3:3b, este comando nos permite descargar e iniciar una sesión que será interactiva con el modelo de IA de Ministral 3b. Al ejecutar el comando, dejamos que se descargue todo lo necesario. Hasta que nos aparezca el mensaje en la terminal de: Send a message (/? for help).

Figura 7. Escribir un mensaje al modelo



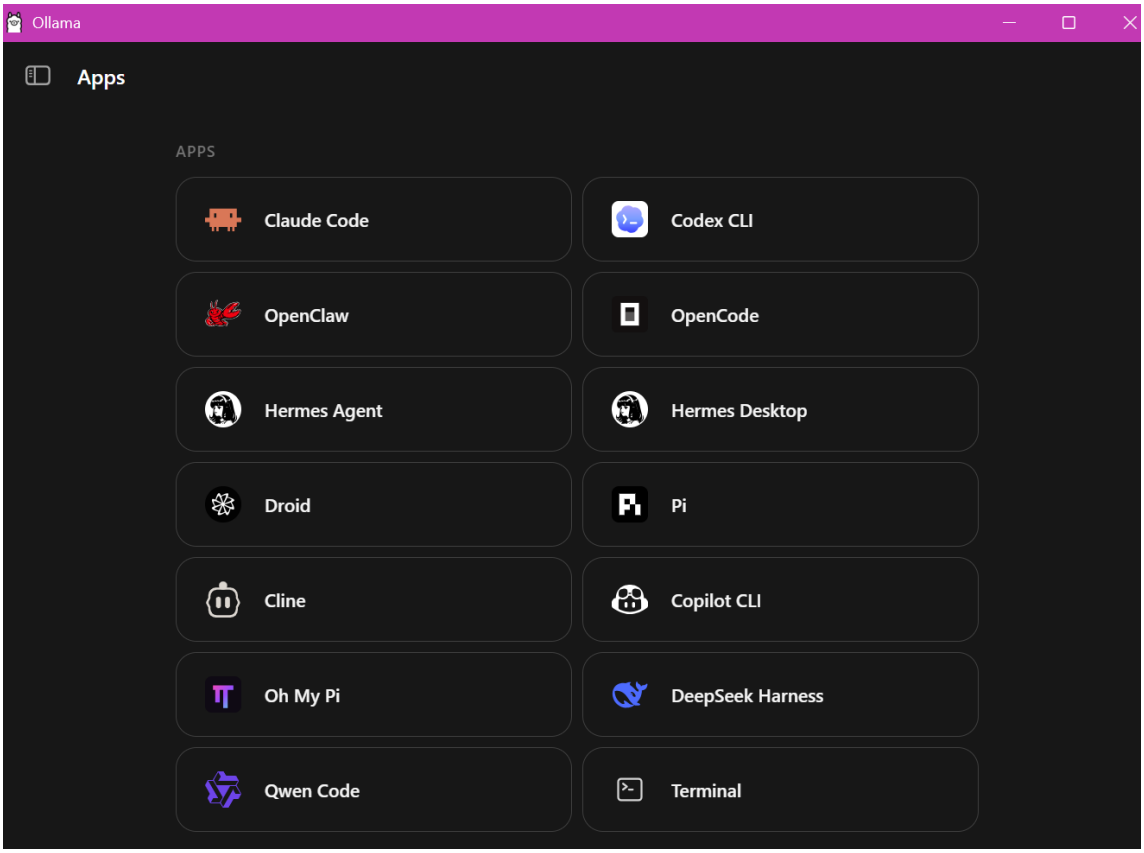
Se le escribe un mensaje al modelo, para comprobar que realmente está funcionando y que es el modelo correcto. En este caso, se le pidió que confirmar que era el modelo Ministral 3B y que, si estaba listo, el modelo respondió de manera exitosa, al tener la confirmación se puede salir del modo interactivo con: /exit.

Figura 8. Comprobar características del Ollama



En la ventana de powershell, se ejecuta el comando de: ollama show ministral-3:3b, esto para ver las características del modelo, que podemos ver que cumple con los requistos de la tabla del proyecto y con las características del equipo donde se está trabajando.

Figura 9. Ollama listo



Una vez terminado todos los pasos anteriores, la ventana de Ollama va a lucir como en la figura 9. Ya está listo para trabajar.

La justificación de porque se seleccionó este modelo de Ministral 3 3B, que de acuerdo a las figuras presentadas anteriormente que este modelo está identificado en Ollama como ministral-3:3b. Va a raíz de las características que van más acorde a los recursos que ofrece el equipo y los objetivos del proyecto, el equipo que se utiliza como se mencionó anteriormente, solo cuenta con 8 GB de memoria RAM, su procesador Intel Core i5-10210U y con únicamente una GPU integrada, no cuenta con una memoria de video dedicada, por estos motivos, no es conveniente trabajar con modelos más grandes, Ministral 3 3B se ajusta perfecto al equipo.

Por otro lado, el tamaño, la selección corresponde a la categoría 3B, sin embargo, Ollama reporta 3.8 mil millones de parámetros, la elección permite que se pueda evaluar tareas como generación de texto, corrección, traducción, resumen, programación e inclusive razonamiento, por otra parte, la cuantización, como se vio en la figura 8, el modelo utiliza Q4_K_M. Una técnica que va a reducir la precisión numérica, que trata de los pesos para poder disminuir lo que es el almacenamiento y memoria necesaria, además, se cumple con la configuración recomendada en la guía de trabajo para los equipos de 8 GB de RAM y que no cuentan con una GPU dedicada.

Por lo tanto, el modelo que se ha seleccionado para el trabajo, permite estudiar las posibilidades como las limitaciones de las ejecuciones de un modelo de lenguaje de manera local teniendo recursos de hardware limitados.

Ahora, se muestra una tabla comparativa con otros dos modelos abiertos, Qwen3 4B y Gemma 3 4B.

Tabla 3. Tabla comparativa de modelos

Características	**Ministral 3 3B**	**Qwen3 4B**	**Gemma 3 4B**
Proveedor	Mistral AI	Alibaba Cloud	Google
Parámetros	3 mil millones	4B	Parámetros de embeddings 675M, parámetros no relacionados con embedding 3,209M
Ventana de contexto	256K tokens	32.768 tokens (nativa)	128K tokens
Licencia	Apache 2.0	Apache 2.0	Gemma Terms of Use
Memoria requerida	16 GB de VRAM en BF16, con menos 8 Gb de RAM/VRAM con cuantización	Aproximado 2,92 GB de memoria GPU, cuatización AWQ-INT4	2,6 GB, sin caché KV. 7,3 GB con caché KV

Benchmark publicado	Arena Duro: 0,305	MMLU-Pro: 50,58	MMLU-Pro: 43,6

De acuerdo a la comparación, se puede observar que los tres modelos, tiene diferencias en sus características, los apartados de cantidad de parámetros, ventana de contexto y consumo de memoria, son datos que siempre son importantes a la hora de seleccionar un modelo para ejecuciones locales. Los resultados de los benchmarks permite que se conozcan los desempeños en distintas tareas, pero, deben tener una interpretación considerando la variante evaluada y condiciones de cada prueba.

Suite de pruebas funcionales

Se evaluó el modelo Ministral 3 3B, esto por medio de diez prompts que están distribuidos en cinco categorías, generación de texto, resumen, corrección o traducción, código, razonamiento y veracidad. La elaboración de la suite se hizo con el modelo Ministral disponible en Ollama, es decir, se le pidió al modelo mediante el modo interactivo que generara el prompts, se guardó en la carpeta prompts creada en visual studio code.

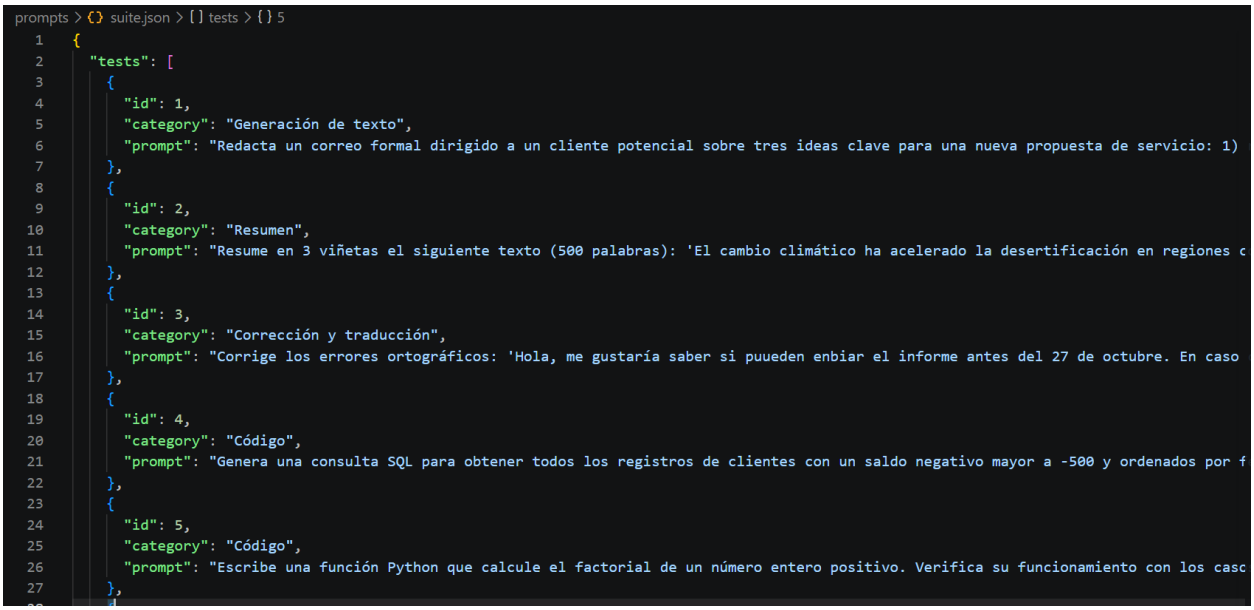
Figura 10. Crear prompt

```
PS C:\Users\Usuario1>  ollama run ministral-3:3b
>>> Actúa como in ingeniero de pruebas de IA, genera un archivo JSON llamado suite.json que contenga exactamente 10 prompts de prueba
...   distribuidos en 5 categorías (2 por cada categoría: Generación de texto donde redacta un correo formal a partir de 3 ideas, res
... umen donde resume un texto de unas 500 palabras en 3 viñetas, corrección y tradución donde corrige ortografía o traducir español
... a inglés, código donde generar una consulta SQL o una función Python y verificar si funciona, y por ultimo razonamiento y veracid
... ad donde hay un problema lógico y una pregunta factual con respuesta verificable). El formato debe ser una lista bajo la clave te
... st, donde cada elemento tenga id (número de 1 al 10), category y prompt
Aquí tienes el archivo `suite.json` con los 10 prompts de prueba distribuidos según las categorías solicitadas:

```json
{
  "test": [
    {
      "id": 1,
      "category": "Generación de texto",
      "prompt": "Redacta un correo formal dirigido a un cliente potencial sobre tres ideas clave para una nueva propuesta de
servicio: 1) reducción de costos en logística, 2) mejora en la experiencia del cliente con un chatbot personalizado, y 3)
implementación de un sistema de seguimiento en tiempo real para pedidos."
    },
    {
```
```

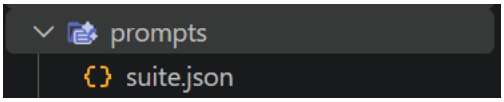
En powershell se ejecuta el comando: ollama run ministral-3:3b, para poder entrar al modo interactivo, y cuando nos salga el comando para agregar texto como se vio en la figura 6 de la instalación del modelo, algo importante, es que se debe tener ollama abierto en segundo plano antes de ejecutar el comando. Vamos a escribir un mensaje para que Ministral-3:3b nos cree el prompt, como el que se muestra en la figura 10, especificando que debe contener dicho prompt. Una vez que termine de ejecutarlo todo, vamos a copiar y pegar el prompt en la carpeta "prompts" creando un archivo nuevo adentro de dicha carpeta.

Figura 11. Pegar prompt en Visual Studio Code



Una vez pegado el prompt en visual studio code, se debe guardar el archivo como: suite.json, adentro de la carpeta prompts. Y este ya queda listo para ejecutarse en cualquier momento.

Figura 12. Carpeta prompts y suite.json



La figura 12, demuestra cómo debe lucir la carpeta prompts y cómo debe lucir el suite.json dentro de la carpeta. Para confirmar que se guardó de manera correcta.

La ejecución del suite.json, el resultado que este daba se calificó las respuestas mediante cuatro categorías:

- Correcta: Cumple con lo que se le solicito.
- Parcial: Solo se cumple una parte de la instrucción o bien, la repuesta está incompleta.
- Incorrecta: Presenta errores, no resuelve de manera adecuada la tarea.
- Alucinación, La información que da es falsa, sin sustento como si fuera cierta.

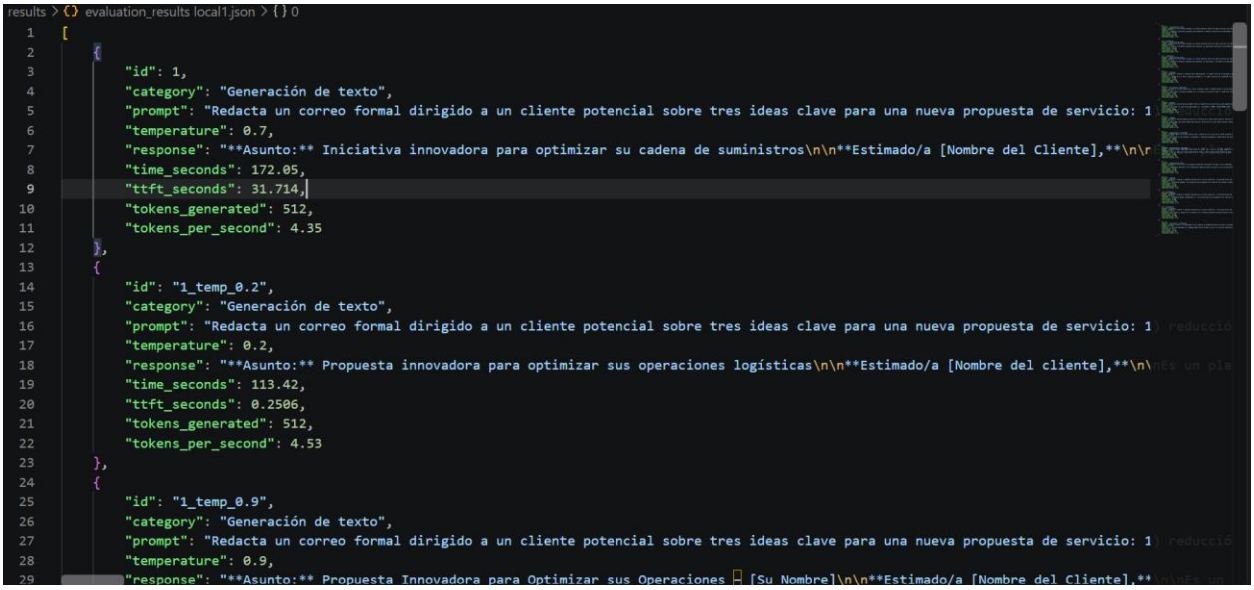
La evaluación del resultado a las respuestas quedo de la siguiente manera:

Tabla 4. Evaluación de resultados

ID	Categoría	Calificación	Justificación
1	Generación de texto	Parcial	El correo formal desarrolla las tres ideas, sin embargo, la repuesta quedo incompleta.
2	Resumen	Correcta	El resumen cumple con ser resumido en tres viñetas, que trataba del cambio climático. Sin embargo, el texto original no contiene 500 palabras.
3	Corrección	Parcial	El mensaje presenta correcciones adecuadas, pero la explicación adicional tiene una observación sobre la palabra antelación.
4	Código SQL	Incorrecto	No logro de manera exitosa cumplir con la consulta, ya que,

			incluye condiciones de saldo que no corresponden a lo que se solicitó.
5	Código Python	Parcial	La función generada parece adecuada, sin embargo, la salida está incompleta, además, no se registra una ejecución que logre confirmar que funciona.
6	Razonamiento lógico	Incorrecto	El modelo logra reconocer que hacen faltas datos, para poder determinar una solución, sin embargo, comienza a mencionar datos que no están en el enunciado y no se llega a una solución coherente.
7	Veracidad factual	Alucinación	Hay afirmaciones sobre la población urbana que son contradictorias y, además, no tienen sustento necesario.
8	Generación de texto	Parcial	El correo técnico identifica lo que es los tres problemas solicitados, a su vez propone acciones, sin embargo, no registra respuesta completa.
9	Resumen	Correcta	Logra resumir las ideas principales, logra mantener la incertidumbre del texto original.
10	Traducción	Correcta	Logra conservar el significado del texto principal en inglés.

Figura 13. Resultado del prompt



En la figura13, se deja una muestra de cómo se ve los resultados a la hora de la ejecución.

Se debian repetir dos prompts con temperaturas de 0.2 y 0.9, por lo que se hizo que se repitieran uno en la generación de texto y otro en el de resumen, según los resultados de la comparación de temperaturas a la hora de revisar, el correo formal tiene diferencias en la estructura, su extensión y los detalles incluidos, la repuesta con temperatura 0,2 desarrollo lo que fueron las tres ideas mediante apartados y listas, por otro lado, la temperatura de 0,9 incorpora ejemplos, así como explicaciones adicionales. Sin embargo, ambas respuestas quedan a la mitad en el registro evaluado, por lo tanto, no se puede determinar únicamente a partir de los resultados cuál sería la más completa.

En el resumen sobre los micro plásticos, las tres respuestas logran conservar la idea principal, mantienen el formato de las tres viñetas solicitado, la diferencia se observa principalmente en la redacción y la forma de como la información se presenta. Por lo tanto, los resultados muestran que la temperatura puede influir en la manera de cómo se redacta las

respuestas, pero, las pruebas realizadas no logran concluir que una temperatura sea más precisa que otra.

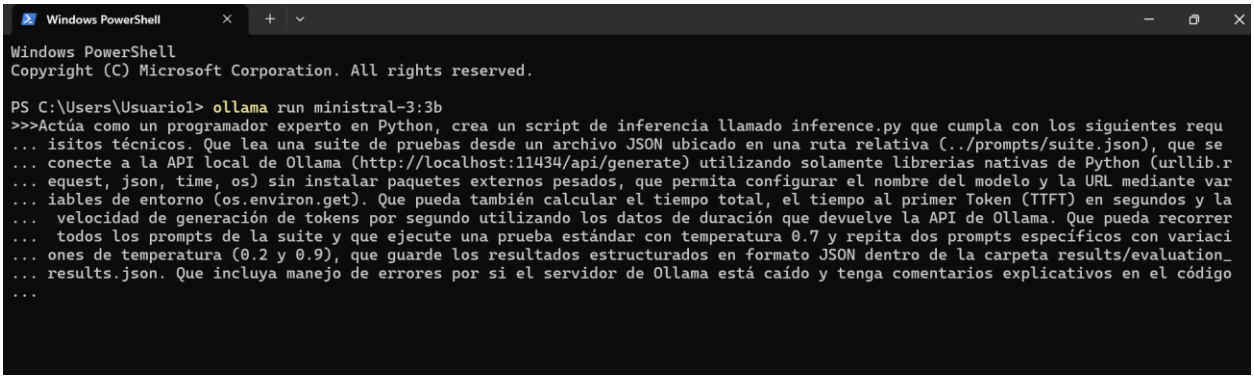
De acuerdo a lo visto anteriormente en este apartado, se pudo mostrar que el modelo muestra resultados satisfactorios en lo que son tareas lingüísticas, así como resúmenes, corrección y traducción, pero, también se pudo observar, que tiene limitaciones en la generación de código y la parte de razonamiento lógico. Por esto, se recomienda revisar las respuestas antes de utilizarlas, específicamente, cuando se tratan de respuestas que se tratan de código o información factual.

Script propio de inferencia en Python

El script de inference.py, nos va a permitir ejecutar pruebas, esto sobre el modelo Ministral 3 3B que hemos venido trabajando mediante la API local de Ollama, para la elaboración del script, se utilizó como base una versión generada que se le pidió al propio modelo por medio de Ollama, en el modo interactivo, sin embargo, debido a que algunas versiones generadas presentaban limitaciones en respuestas incompletas, como errores en el código, se tomó la acción de revisar con Gemini los errores que habían que modificar.

Se carga los prompts desde un archivo JSON, se envía cada solicitud al modelo, luego de esto al terminar, se recopila las respuestas en una carpeta llamada results, junto a las métricas de ejecución. Vamos a ver algunas figuras de cómo va la ejecución y el script creado.

Figura 14. Creación del script inference.py



En la terminal de powershell, con Ollama abierto en segundo plano, se va a ejecutar el comando en la terminal de powershell: ollama run ministral-3:3b, para entrar al modo interactivo y se escribe cuando salga el signo de: > > >, acá se le pide al modelo que actúe como un programador de Python, dándole las características que debe tener el código de inference.py. Y se espera que se genere el código.

Figura 15. Script creado

Aquí tienes el script `inference.py` que cumple con todos tus requisitos técnicos:

```python
import os
import json
import urllib.request
import urllib.parse
import time
from datetime import datetime
import sys

def load_suite_from_json(suite_path):
    """Carga la suite de pruebas desde un archivo JSON."""
    try:
        with open(suite_path, 'r') as f:
            return json.load(f)
    except Exception as e:
        raise ValueError(f"Error al cargar el archivo de suite: {e}")
```

Una vez que el modelo genere el código, se va a copiar y pegar en un archivo dentro de la carpeta src creada en visual studio code, es importante aclarar que el modelo puede dar partes de

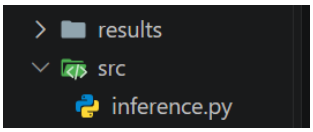
código incompletas, con errores o no generarlo completo como se mencionó, por esto, es importante revisarlo para poder verificar que se debe editar en la base creada por el modelo.

Figura 16. Script en visual studio code

```
src > inference.py > ...
1   import json
2   import urllib.request
3   import urllib.error
4   import time
5   import os
6   import sys
7   import argparse
8
9   # Se obtiene la carpeta donde se encuentra el scritp
10  #Se construye las rutas de los archivos del proyecto
11  SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
12
13  def load_suite(suite_path):
14      """Carga la suite de pruebas desde un archivo JSON."""
15      try:
16          #Se abre el archivo, se convierte el contenido JSON en datos de Python
17          with open(suite_path, "r", encoding="utf-8") as f:
18              return json.load(f)
19      except Exception as e:
20          #Se informa si ocurre un error a la hora de abrir o leer el archivo
21          print(f"[Error] No se pudo cargar el archivo de suite en '{suite_path}': {e}")
22          return None
23
24  def query_ollama(api_url, model_name, prompt_text, temperature, max_tokens):
25      """
26      Envía una petición POST a la API local de Ollama y calcula
27      las métricas de rendimiento exigidas (TTFT y tokens/s).
28      """
29      #Prepara los datos que se envian a la API de Ollama
```

Se pega el código en el archivo creado adentro de la carpeta src, para guardar el script cómo: inference.py, se le agregan los comentarios necesarios para guiarse y la edición necesaria al código si hay errores.

Figura 17. Carpeta src con el script de inference.py y crear carpeta results



En la figura 17, se ve cómo debe de lucir una vez guardado el archivo inference.py dentro de la carpeta src, es importante antes de hacer ejecuciones con el inference.py, crear la carpeta llamada results, porque acá es donde se guarda los resultados de las ejecuciones.

Figura 18. Ejecución del inference.py de manera local

```
PS C:\Users\Usuario1\Mis documentos\octavo cuatri\PROGRAMACIÓN DE LENG.DE 4
TA GENERACION\Proyecto-mistral> cd src
PS C:\Users\Usuario1\Mis documentos\octavo cuatri\PROGRAMACIÓN DE LENG.DE 4
TA GENERACION\Proyecto-mistral\src> py inference.py
=== Iniciando evaluación local ===
Modelo: ministral-3:3b | Temperatura: 0.7 | Max Tokens: 512

[Ejecutando] Test ID 1 [Generación de texto]
 -> Tiempo total: 172.05s | TTFT: 31.714s | Tokens/s: 4.35
```

En visual studio code se habrá la terminal y se mantiene Ollama abierto en segundo plano, una vez en la terminal de visual, se escribe el primer comando de: cd src, esto nos permite ubicarnos en donde se encuentra el inference para ejecutarlo. Una vez que veamos que nuestro PS C terminal en src, escribimos el segundo comando: py inference.py, y esperamos que inicie la evaluación local, la duración de la ejecución depende de la velocidad del equipo y en tener otras ventas cerradas, para las ejecuciones de forma local, se recomienda tener solo Ollama abierto y visual studio code para las ejecuciones. En la figura 18, podemos ver que nos muestra la primera variación de temperatura de 0.2 y 0.9.

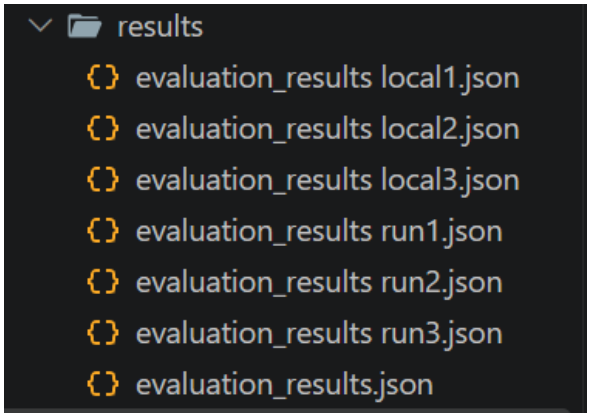
Figura 19. Ejecución exitosa del inference.py de forma local

```
[Ejecutando] Test ID 10 [Corrección y traducción]
 -> Tiempo total: 5.77s | TTFT: 2.2118s | Tokens/s: 4.32

[✓] Evaluación finalizada con éxito. Resultados guardados en: C:\Users\Usua
rio1\Mis documentos\octavo cuatri\PROGRAMACIÓN DE LENG.DE 4TA GENERACION\Pr
oyecto-mistral\src\..\results\evaluation_results.json
PS C:\Users\Usuario1\Mis documentos\octavo cuatri\PROGRAMACIÓN DE LENG.DE 4
TA GENERACION\Proyecto-mistral\src>
```

Una vez que termine la ejecución local de inference.py, nos saldrá que la ejecución fue exitosa, demostrando los mensajes que tiene el inference para que el usuario pueda ir entendiendo el procedimiento, además, muestra la ruta en donde se guarda el resultado, en este caso, la carpeta que se creó llamada results, el resultado se guarda bajo el nombre de evaluation_results.json.

Figura 20. Carpeta results con resultados de la ejecución del inference.py



En este caso, podemos ver que la carpeta results cuenta con 7 evaluation_results.json, sin embargo, esto es porque debió a las pruebas que pedía el proyecto se guardaron copias para que no se sobre escribiera varias veces el mismo archivo, de esta forma conservar los resultados para los demás apartados del trabajo. En results siempre existirá solo un resultado, el cual es: evaluation_results.json, este es el archivo que se sobre escribe cada que hagamos ejecuciones.

Ahora hablando sobre los parámetros de ejecución, el script permite configurar su funcionamiento, mediante lo que son los argumentos de línea de comandos:

Tabla 5. Línea de comandos

Parámetro	**Función**
model	Esto especifica el modelo de Ollama que se está utilizando.
suite	Indica lo que es la ruta del archivo JSON que básicamente contiene los casos de prueba.
temperature	Esto nos ayuda a definir la temperatura utilizada para generar las respuestas.
max-tokens	Va establecer el número máximo de tokens que se generarán por respuestas.
url	Es la dirección de la API de Ollama

Estos parámetros ayudan a poder modificar la configuración de las pruebas, de este modo, no se tiene que cambiar directamente el código fuente. Por otro lado, tenemos los resultados que se almacenan en results/evaluation_results.json, para cada prueba el archivo va a registrar la información como el identificador, también como la categoría, el prompt, temperatura que está utilizando, las respuestas generadas/obtenidas, el tiempo total de la ejecución, tokens generados y, además, velocidad de generación en tokens por segundo. El script generado, permite registrar un valor denominado TTFT, sin embargo, es importante mencionar que la implementación actual del script, utiliza lo que es promt_eva_duration, esto corresponde al tiempo de procesamiento del prompt y no realmente al tiempo real transcurrido hasta la generación del primer token, así que no se puede tomar este valor como una medición precisa del TTFT. Lo que es la velocidad de generación, se basa en un cálculo a partir de la cantidad de tokens generados, el tiempo de generación informado por el mismo Ollama, por lo que, las métricas permiten observar el comportamiento del modelo durante las pruebas. Es importante destacar que se deben interpretar considerando las limitaciones de la implementación.

El script tiene módulos incluidos que sería la biblioteca estándar de Python, que fueron json, urllib, time, os, sys y argparse, por lo que no fue necesario instalar dependencias externas, para el funcionamiento del código, además, se tomó en consideración que el equipo ya trabaja más lento. El archivo requirements.txt que sale en la raíz del proyecto solo documenta esta misma situación, por otro lado, las rutas de los archivos se construyen por medio de la ubicación del propio script, así que no depende de una ruta absoluta fija. Por otra parte, el programa logra completar errores, ya sean de la conexión con Ollama, las respuestas del JSON que no logran ser validas o bien otros errores inesperados, todo esto se muestra con mensajes en la terminal para que el usuario pueda saber que paso.

Contenerización con Docker

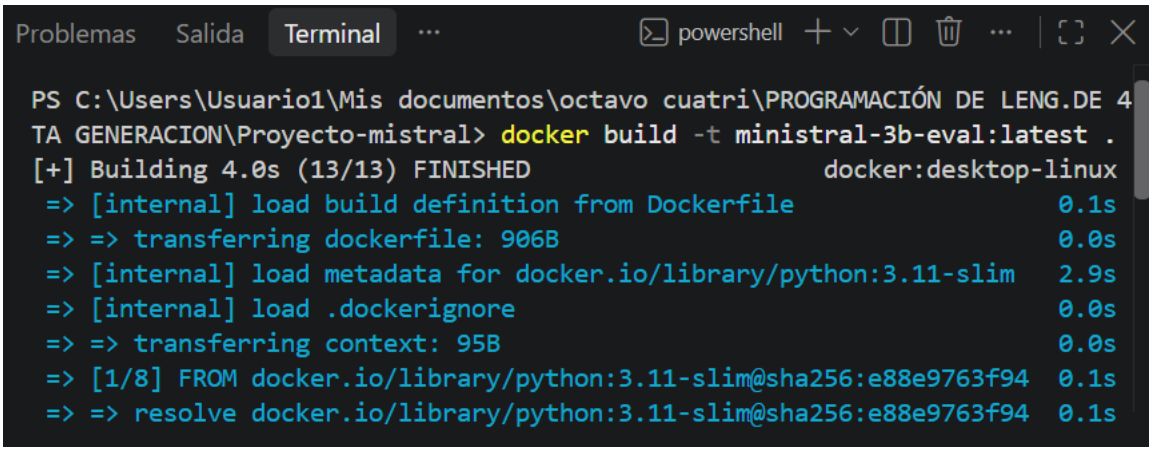
La configuración del contenedor, se basó en la creación de un Dockerfile, con la imagen oficial Python:3.11-slim esto para poder ejecutar el script de inferencia src/inference.py en conjunto con la suite de pruebas ubicada en la carpeta prompts, se aplicaron lo que son las buenas prácticas, la configuración incorpora una imagen de base ligera con su versión definida de Python, además, se organiza la instalación de dependencias antes de copiar el código fuente, esto para poder aprovechar la caché de Docker, por otro lado, utiliza un usuario no administrador. También se creó el archivo ".dockerignore" que sirve para excluir archivos innecesarios, tales como: .git/, docks/, results/ y lo que son archivos temporales de Python.

Lo que se trata de la gestión del modelo y su ejecución, los archivos del modelo no están incluidos en lo que es imagen Docker, de esta manera se evita aumentar el tamaño, en lugar de esto, el script se comunica con lo que es el servidor de Ollama del equipo mediante lo que es la API, la ejecución del Docker, se realizó con Docker run, para esto se configuro la dirección de Ollama y se le vinculo la carpeta local results con /app/results mediante un volumen, se agregó

también, el –rm, esto elimina el contenedor al finalizar, más adelante veremos una figura con esta ejecución.

Por otra parte, se logro poder observar que el modelo ministral-3:3b se ejecuta utilizando CPU, manteniendo un tamaño de 2.7 GB y el contexto de 4096 tokens, por otra parte, se logró observar el registro de Docker stats, el cual registro una medición de 22.72% de uso de CPU y también, 18.11 MiB de memoria utilizada por el contenedor, algo importante, es que estas métricas corresponden a las observaciones puntuales y no representan como tal el consumo total de Ollama y el modelo, a continuación, veremos las figuras sobre lo mencionado en estos apartados.

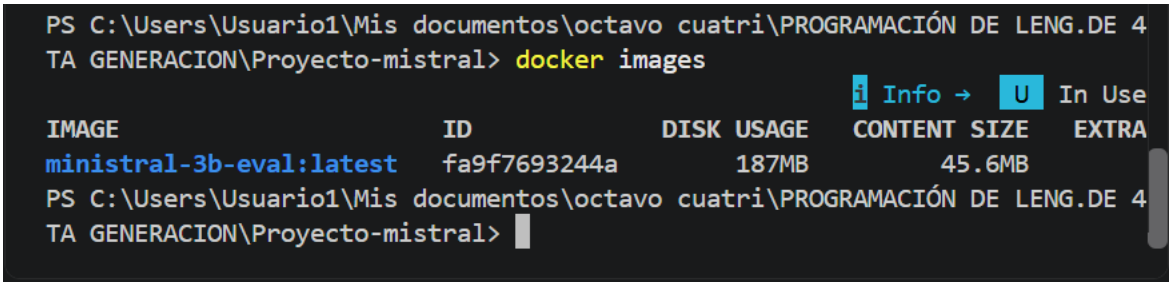
Figura 21. Docker build construido



En visual studio code, se abre la terminal, antes de ejecutar comandos, se va abrir Ollama y Docker Desktop, para que estén abiertos y trabajando en segundo plano. Una vez hecho esto, se va a escribir el comando: Docker build -t ministral-3b-eval:latest .

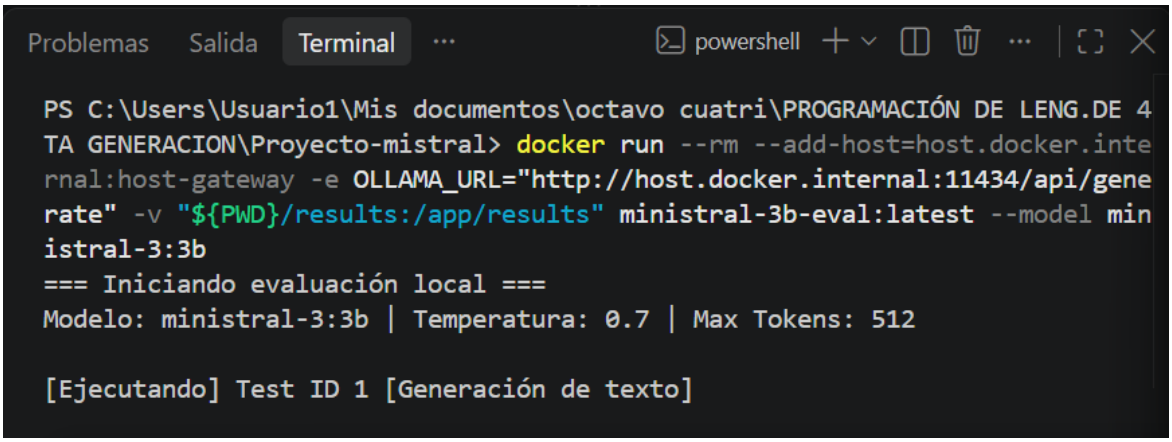
Se espera a que se complete los 13 de 13, y que muestre finished en la terminal.

Figura 22. Verificar images creada



Una vez terminado el Docker build, en la misma terminal de visual, se va a ejecutar el comando: docker images, para poder verificar las características de la imagen, la figura 22, muestra el image que está bajo el nombre ministral-3b-eval:latest, el ID siendo fa9f7693244a, su disk usage que se mantiene en 187MB y content size de 45.6MB.

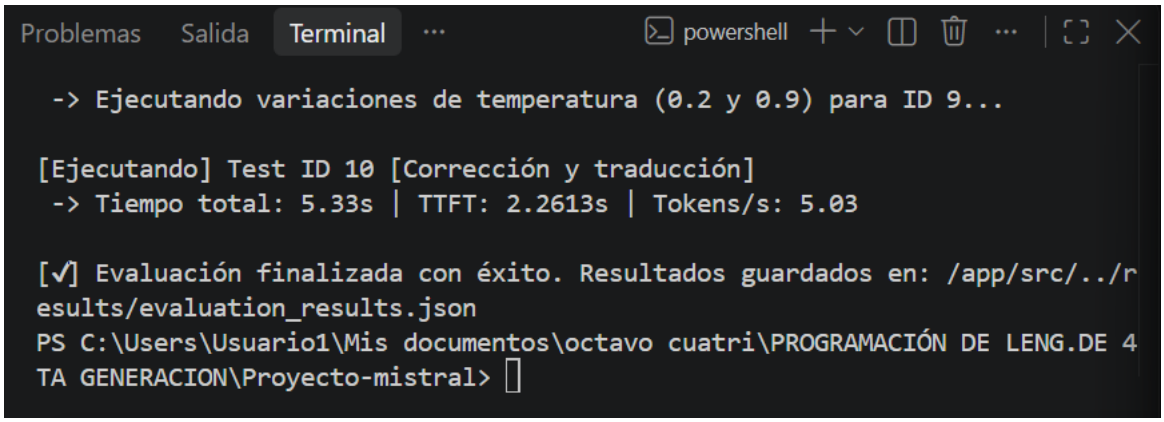
Figura 23. Docker run, ejecuciones



En la misma terminal de visual, recordando que Docker Desktop y Ollama deben estar abierto, se va a ejecutar este comando en la terminal: docker run --rm --add-host=host.docker.internal:host-gateway -e OLLAMA_URL="http://host.docker.internal:11434/api/generate" -v "${PWD}/results:/app/results" ministral-3b-eval:latest --model ministral-3:3b

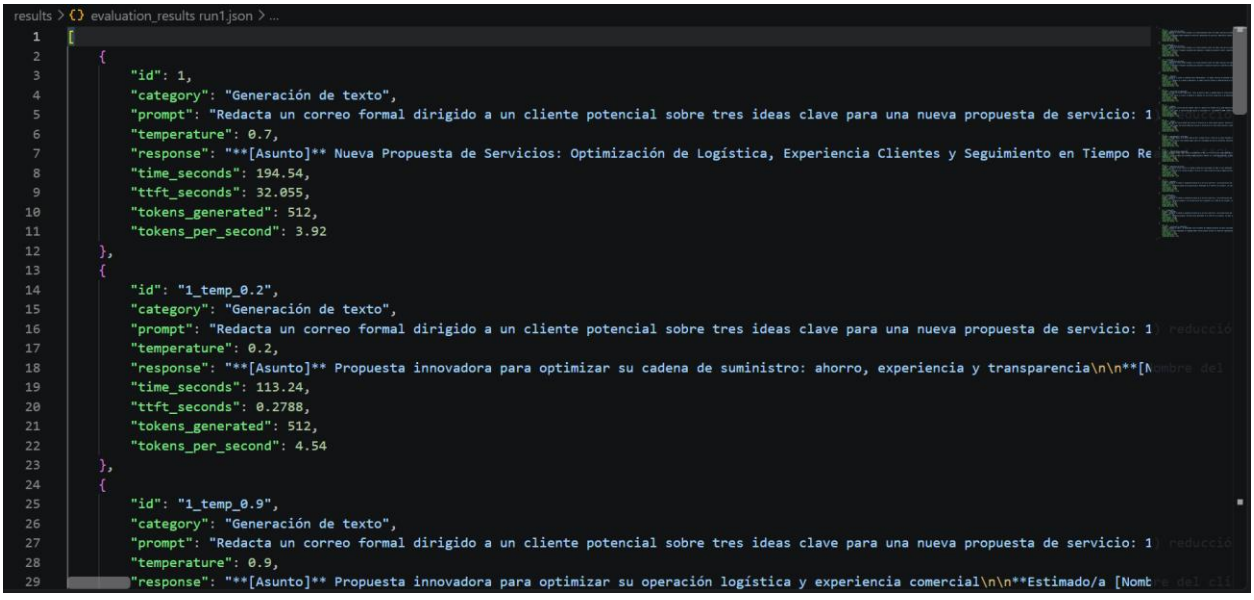
Tal como se muestra en la figura 23, este comando nos permite ejecutar la suite dentro del contenedor, acá esperamos que el docker run termine de ejecutarse, a veces se queda congelado, pero solo se debe dejar que termine y al final muestra todo el procedimiento.

Figura 24. Resultado final de docker run



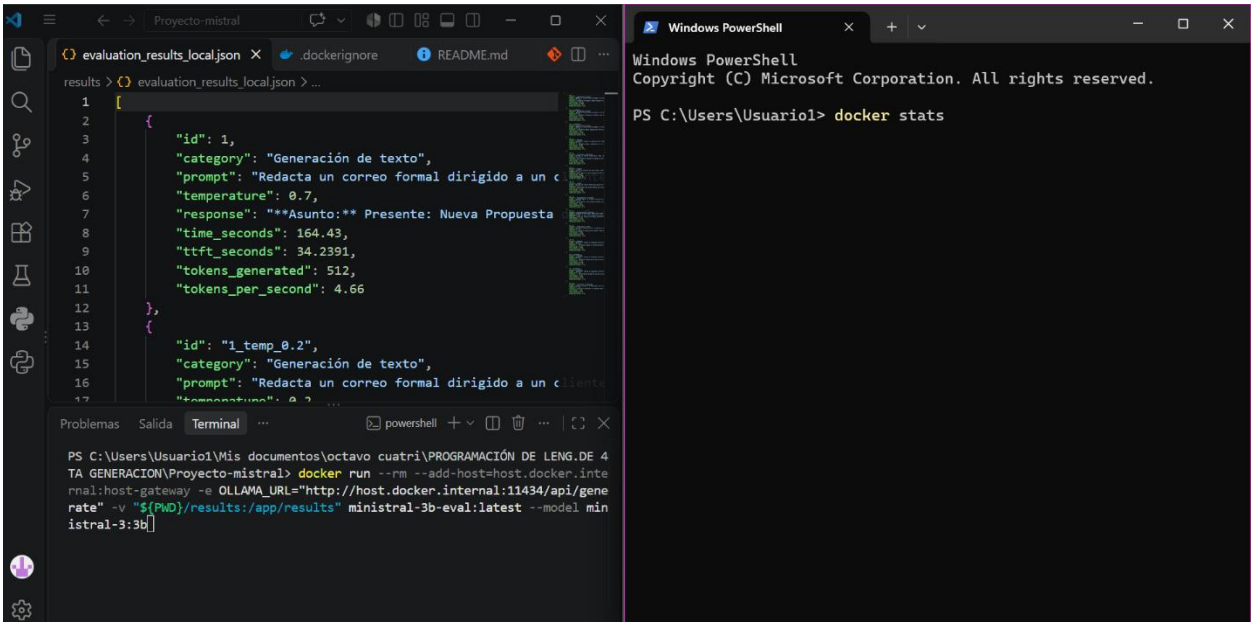
En la figura 24, podemos ver el resultado final del docker run que se mostró en la figura 23, se puede observar que la ejecución fue exitosa y guardada en la carpeta results.

Figura 25. Resultados de la ejecución del docker



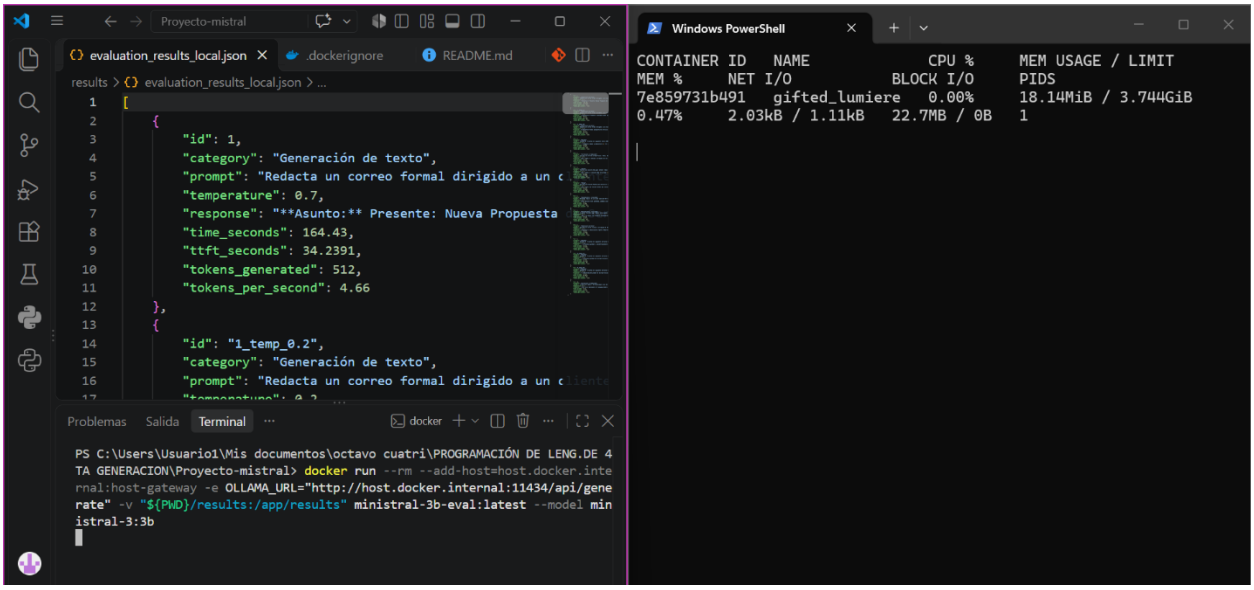
En la figura 25, podemos ver como quedo el resultado del docker run, que se guardó correctamente.

Figura 26. Prueba de medición con docker stats



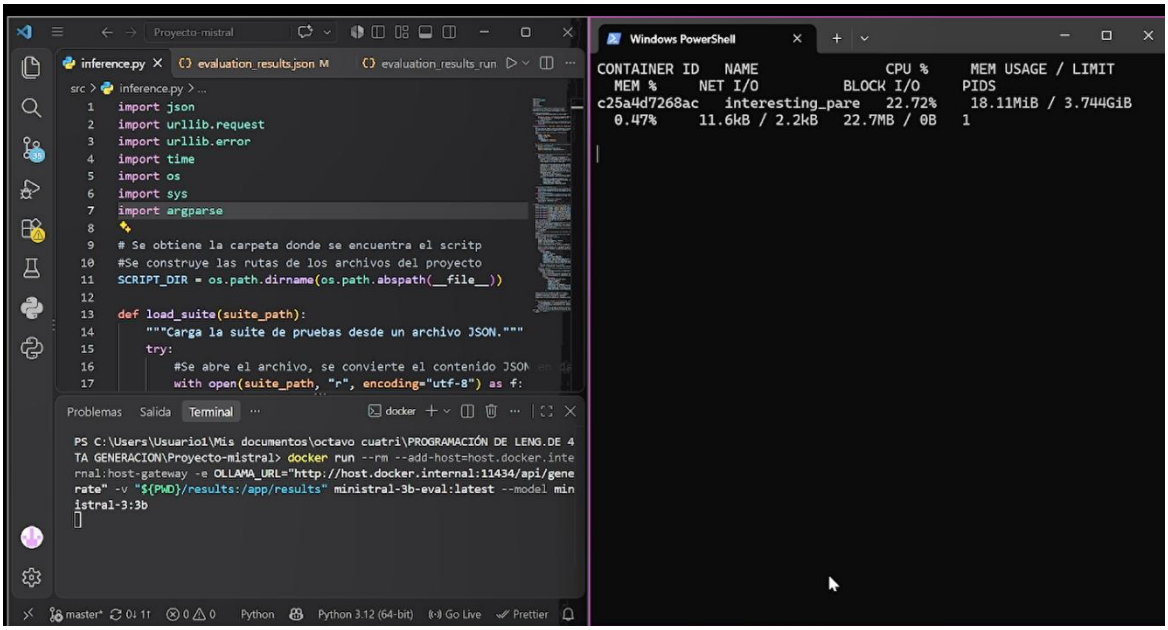
En la terminal de visual studio code, después de ejecutar el primer docker run, se puede volver a ejecutar uno más para hacer la comprobación de la medición. Por lo que, en la terminal de visual studio de vuelve a escribir el comando que ejecuta el docker run y en otra terminal en este caso powershell, escribimos el comando: docker stats, una vez escrito lo necesario en ambas terminales, primero ejecutamos el comando docker run y una vez que comience a correr, ejecutamos el docker stats en la otra terminal.

Figura 27. Monitoreo



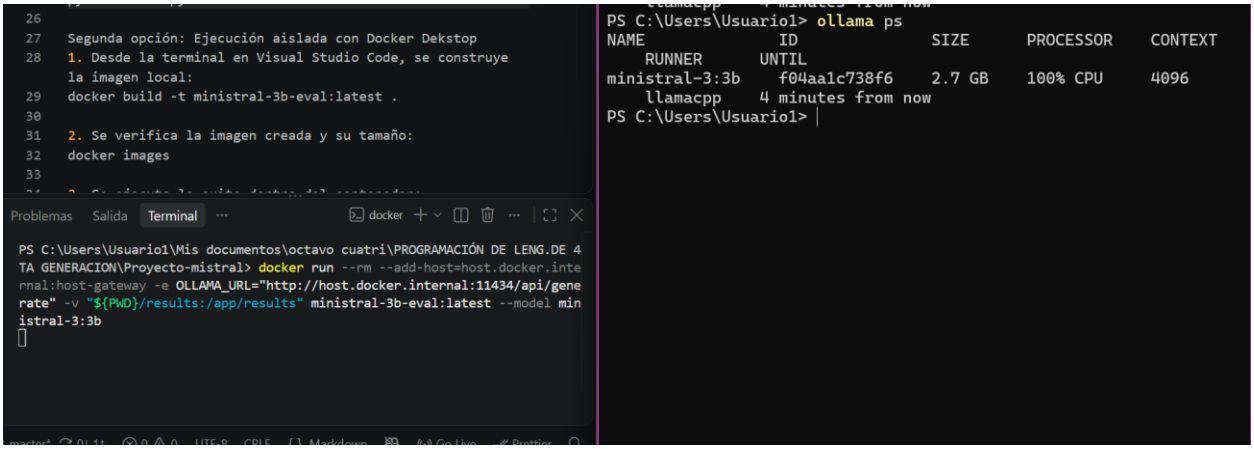
Ambas terminales se van a ver como en la figura 27, para esta parte, se recomienda hacer una grabación de pantalla, para poder ver dónde sale el consumo de CPU y memoria, ya que suele salir de golpe y no se logra apreciar bien.

Figura 28. Consumo de CPU y memoria localizado



En la figura 28, podemos observar el consumo más alto al que llego esta ejecución, que fue de 22.72 % en la CPU, que el Mib y GiB quedo en 18.11 y 3.744.

Figura 29. Ejecución en CPU



Para ver si el modelo ministral-3:3b se ejecuta utilizando CPU, hacemos casi lo mismo como con docker stats, solo con la diferencia de que usaremos ollama ps. En la terminal de visual volvemos a poner el comando docker run y en powershell, ponemos el comando: ollama ps, ejecutamos primero el docker run en visual y nos vamos a la terminal de powershell a ejecutar ollama ps, podemos ver que nos sale que se está utilizando CPU y que tiene un tamaño de 2.7 GB, con 4096 tokens.

Análisis de desempeño

Se ejecutó la suite de pruebas tres veces en las configuraciones de ejecución nativa mediante Ollama y ejecución dentro del contenedor de Docker, para ambas se utilizó el modelo ministral-3:3b, con su cuantización Q4_K_M y temperatura de muestreo de 0,15. Para la evaluación de la suite se calculó el promedio y desviación estándar de las tres ejecuciones para cada métrica, estas comparaciones van a permitir que se pueda observar las diferencias de

rendimiento entre ambas, sin cambiar deliberadamente el modelo utilizado, a continuación se muestra los resultados:

Tabla 6. Resultados de rendimiento

Métrica	**Ollama promedio nativo**	**Desviación estándar**	**Promedio de Docker**	**Desviación estándar**
Tiempo hasta el primer token (s)	6,658	0,101	7,007	0,081
Velocidad de generación (tokens/s)	4,942	0,151	4,585	0,079
Tiempo promedio por caso (s)	70,74	1,20	72,12	3,36
Tiempo total de la suite (s)	707,40	12,04	721,22	33,56

Estos resultados muestran que la ejecución nativa presenta un menor tiempo de promedio hasta el primer token, también que tiene una mayor velocidad de generación, Docker registra un TTFT aproximadamente de 0,349 segundos mayor, con una velocidad de generación aproximada de 7,2% inferior a la ejecución nativa, por otra parte, el tiempo total promedio de la suite fue de 13,82 segundos mayor en Docker. La desviación estándar nos permite observar la parte de la variabilidad entre las tres ejecuciones, en este caso, Docker presenta una variación mayor en el

tiempo total de la ejecución de la suite, mientras que las mediciones de TTFT y su velocidad de generación tuvieron desviaciones estándar algo pequeñas en ambas configuraciones.

Ahora, vamos a ver el uso de CPU y la memoria en Docker, durante las tres ejecuciones, se logró recopilar lecturas de manera puntual de docker stats, mediante grabaciones de pantalla, a partir de las observaciones transcritas se pudo calcular los promedios, así como sus desviaciones estándar:

Tabla 7. Uso de CPU más memoria de Docker

Ejecución	Promedio de CPU	Desviación estándar de CPU	Promedio de RAM	Desviación estándar de RAM
Docker 1	3,16	5,75	18,11	0,00
Docker 2	4,44	7,24	18,20	0,22
Docker 3	6,79	16,37	16,82	4,13

Las cifras dadas anteriormente describen las lecturas de manera puntual registradas, en la tercera ejecución, se dio un caso sobre que la última lectura mostró 62,60% de CPU más adelante veremos esto en las figuras, además mostró 2,46 MiB de memoria, y con cero procesos, esta observación se conserva, sin embargo, puede corresponder a una transición al finalizar el contenedor. Un detalle importante aclarar, es que las mediciones de Docker no deben compararse de manera directa con lo que son los máximos de CPU, así como de memoria registrada en la ejecución nativa, esto debido a que son estadísticas diferentes, el nativo registra máximos, mientras que, por otro lado, en Docker se calculó promedios y desviaciones estándar de muestras puntuales, los máximos registrados de las ejecuciones nativas fueron:

Ejecución nativa	CPU máxima	RAM máxima	Métrica GPU máxima
Nativa 1	97,76	7,84	22,62
Nativa 2	97,39	7,82	15,32
Nativa 3	121,82	7,73	24,56

La CPU puede superar el 100%, y esto es debido a cuando la herramienta suma la utilización de varios núcleos, la métrica de GPU se conserva con el nombre registrado en los datos GPU_Actividad_Suma_Porcentajes en un archivo que se creó para cada ejecución llamado medición local, sin embargo, no se pudo lograr verificar cómo se calcula exactamente, así que no se debe interpretar como el porcentaje global de uso de la GPU.

Por otra parte, vamos a ver los gráficos comparativos, estos van a representar lo que son promedios de ambas configuraciones, así como sus desviaciones estándar.

Figura 30. Gráfico de tiempo

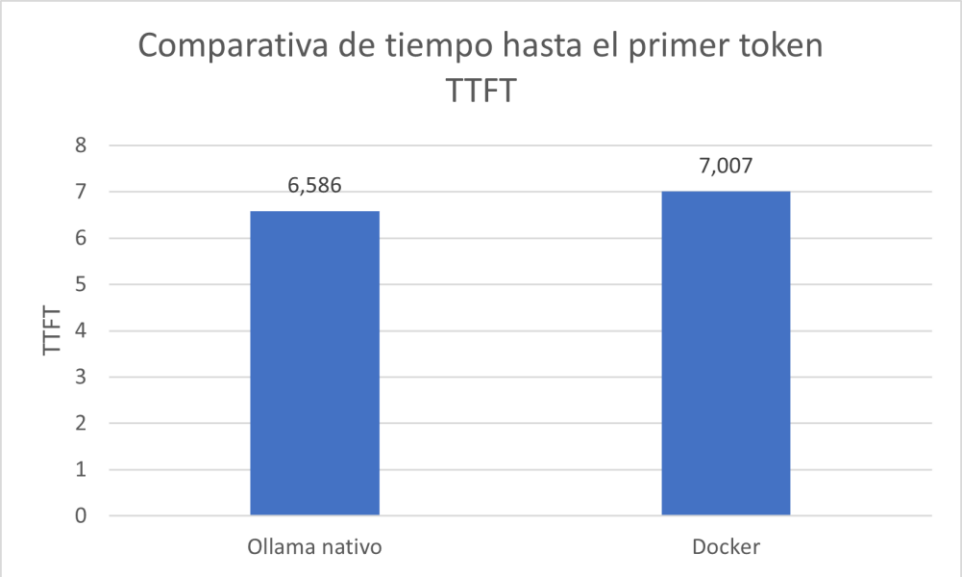


Figura 31. Gráfico de velocidad

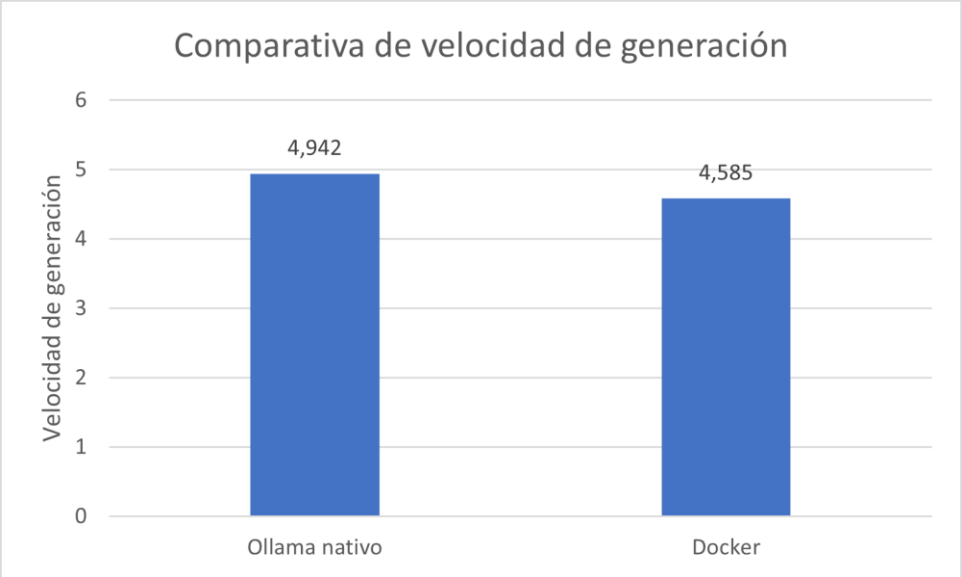


Figura 32. Gráfico de promedio de caso

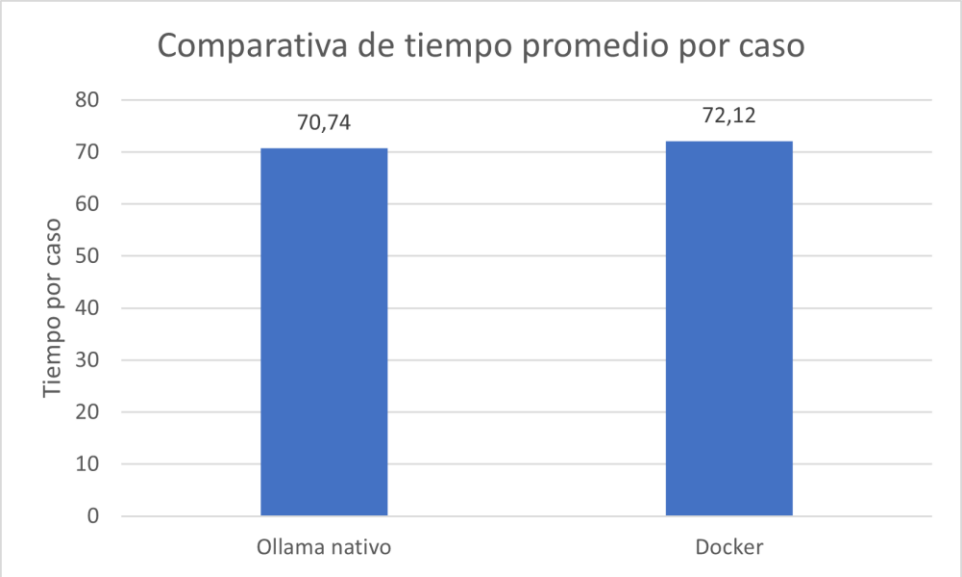
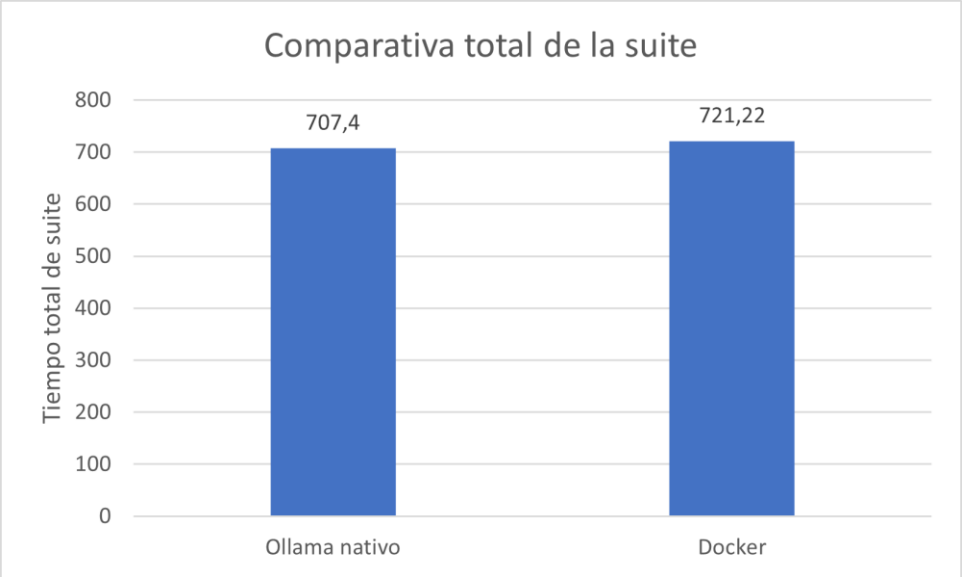


Figura 33. Gráfico de tiempo total de la suite



Las limitaciones prácticas de ejecutar modelos localmente, es que se depende de los recursos disponibles en el equipo, sobre todo de la memoria RAM, procesador y la memoria de la GPU cuando existe, en este caso el proyecto se ejecutó en un equipo con 8 GB de RAM, con gráficos integrados Intel UHD, por este motivo se eligió ministral-3:3b cuantizado en Q4_K_M, para poder reducir los requisitos de memoria. Sin embargo, pese a que la cuantización permite ejecutar modelos con recursos más limitados, esto no elimina el consumo considerable de memoria, ni tampoco va a garantizar una generación rápida, durante las pruebas nativas se pudo registrar un uso máximo de RAM que fue cercano a 8 GB y a la vez, una utilización elevada de CPU, por lo tanto, evidencia que el equipo trabaja cerca de sus límites de memoria durante las ejecuciones.

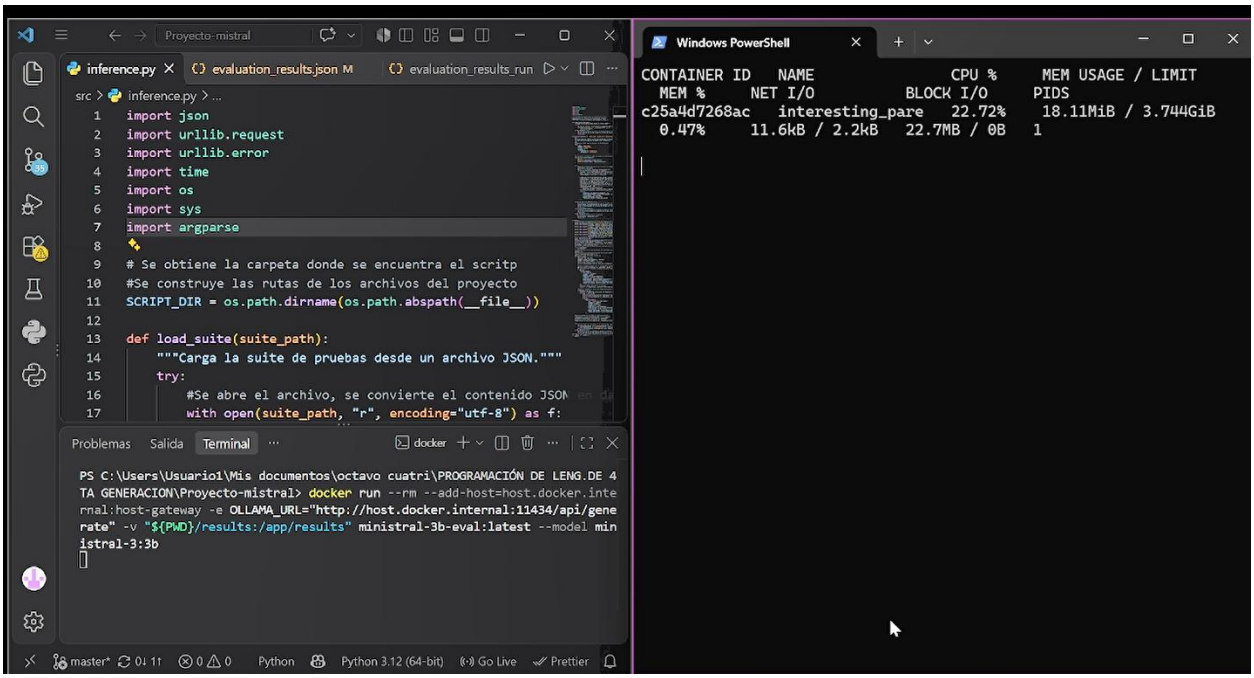
Por otra parte, Docker facilita la reproducibilidad, así como el aislamiento del entorno, aunque introduce una capa adicional de ejecución, en las pruebas, la configuración de Docker mostró velocidad promedio de generación menor, con un tiempo total de la suite algo mayor. Aun así, los resultados no permiten afirmar que Docker sea lento en todo momento, puesto que, el

resultado depende del equipo, así como de configuraciones del contenedor y sus condiciones de ejecución.

Por otro lado, tenemos lo que es la diferencia entre temperaturas, en este caso de dos, la temperatura de muestreo y del hardware, la temperatura configurada en el modelo fue de 0,15, dicho parámetro influye en la selección de tokens, al igual que en la variabilidad de las respuestas generadas, mientras que la temperatura del hardware, se mide en grados Celsius y está representa el calentamiento de componentes, tales como el procesador o la GPU, sin embargo, no se registró datos de esta variable durante las pruebas, por lo tanto, no es posible presentar los resultados sobre el calentamiento del equipo.

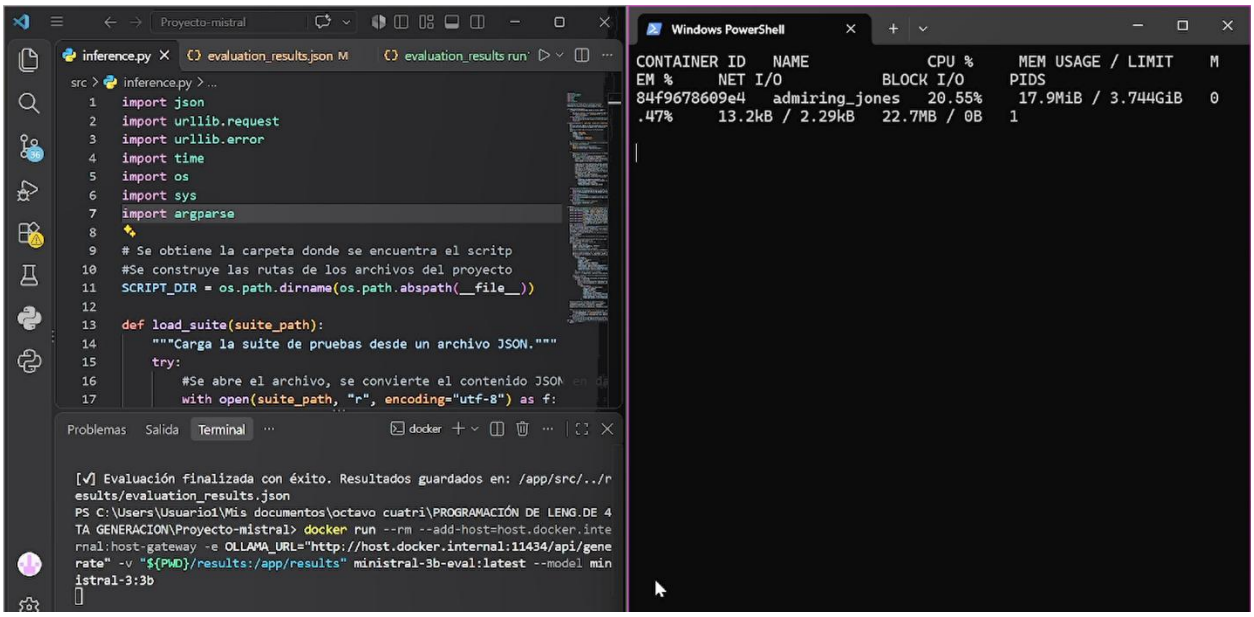
A continuación, se verán algunas figuras que pertenecen a este apartado, que tienen que ver con la relación de las tres ejecuciones nativas como las del Docker:

Figura 34. Primera ejecución de Docker stats



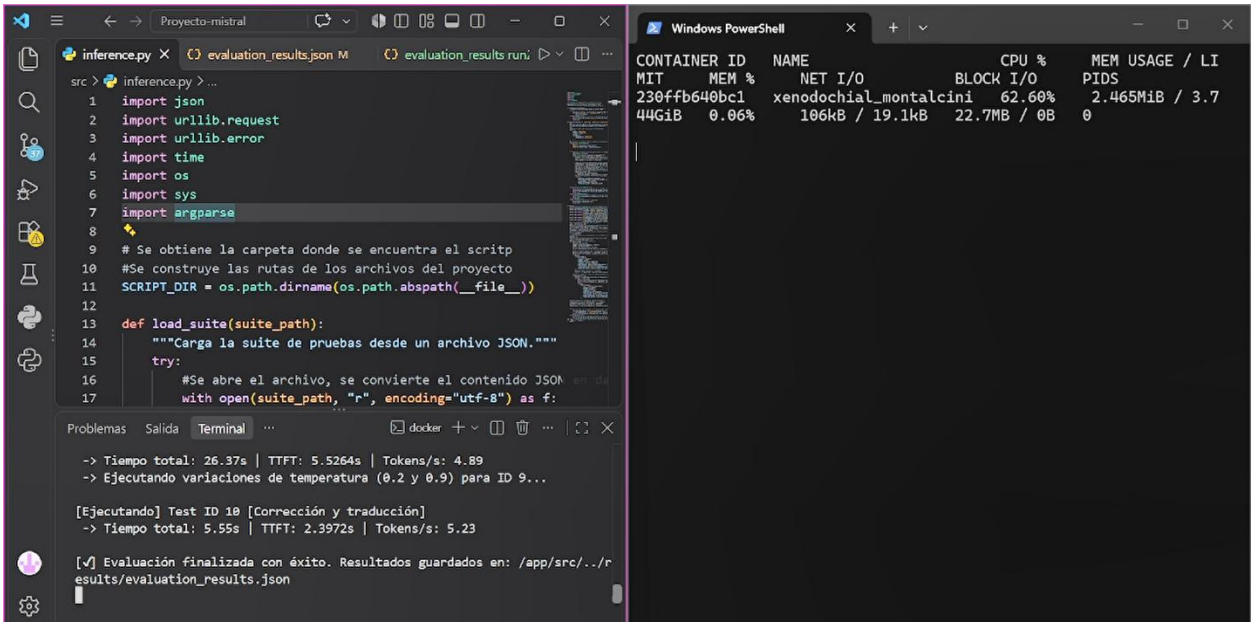
En esta figura, se puede observar cómo se miraba la CPU y la memoria, en donde se nota más consumo. Ya que el consumo iba variando en la grabación, así como de las otras dos ejecuciones.

Figura 35. Segunda ejecución de Docker stats



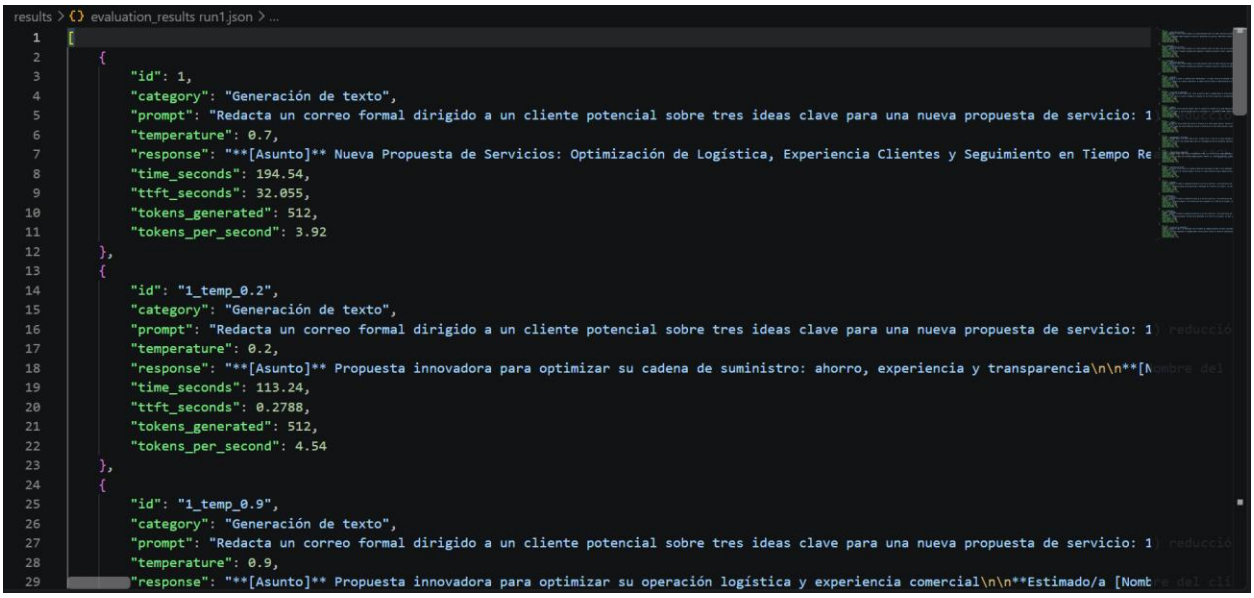
Se puede observar la segunda ejecución, y se puede ver la captura de la grabación donde se muestra el consumo más alto que se apreció.

Figura 36. Tercera ejecución de Docker stats



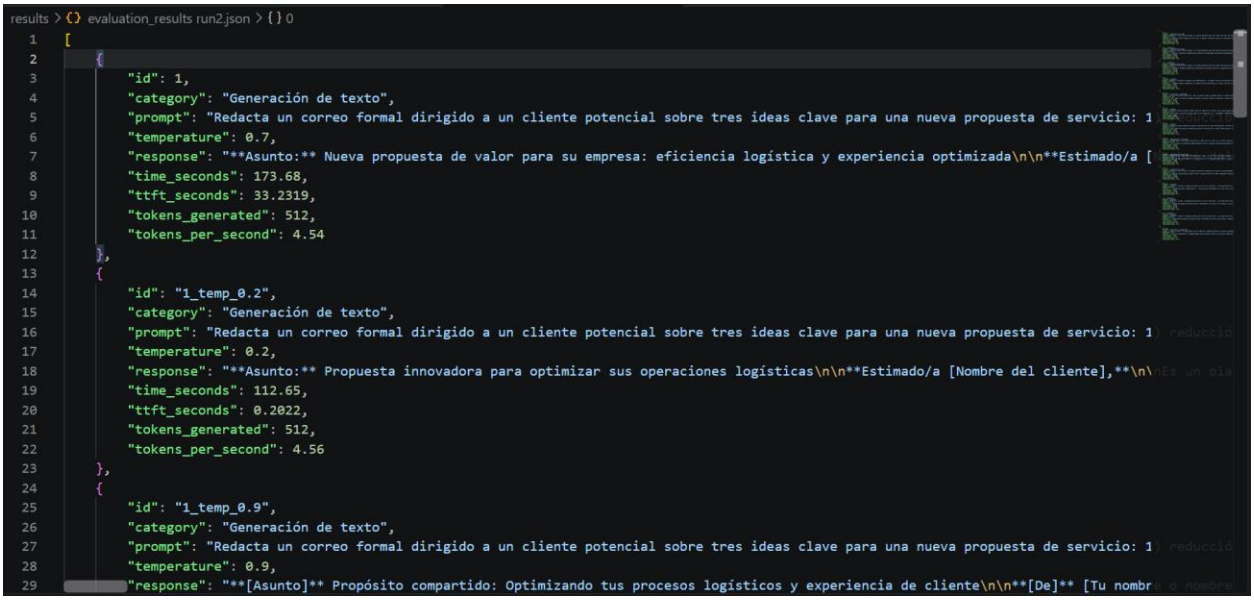
Esta es la última y tercera ejecución del Docker stats, donde se puede apreciar que el consumo más alto, fue llegando prácticamente al final. Que fue lo que se mencionó en los apartados de arriba.

Figura 37. Resultado de la ejecución Docker run 1



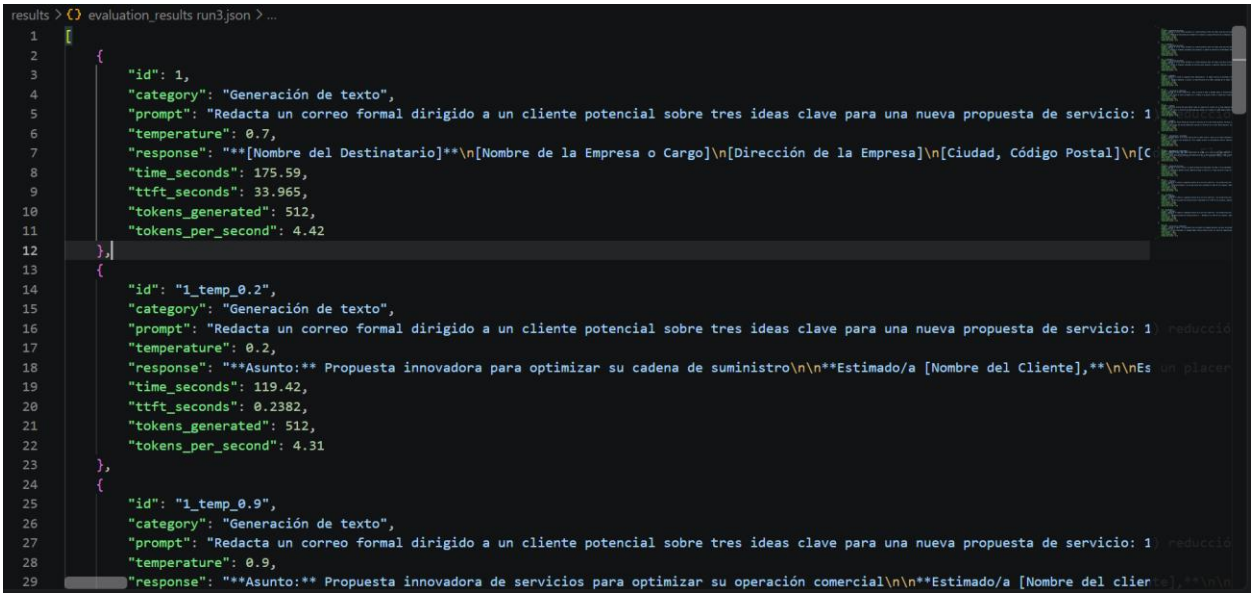
En la figura podemos observar el resultado de Docker run1, este es el resultado de la ejecución de la suite, su evaluation_results.json, antes del json, se agregó el: run1. Para no perder los resultados de las tres ejecuciones y poder ver sus TTFT, junto a lo demás como: time_secondos, tokens_generated y tokens_per_second.

Figura 38. Resultado de la ejecución Docker run 2



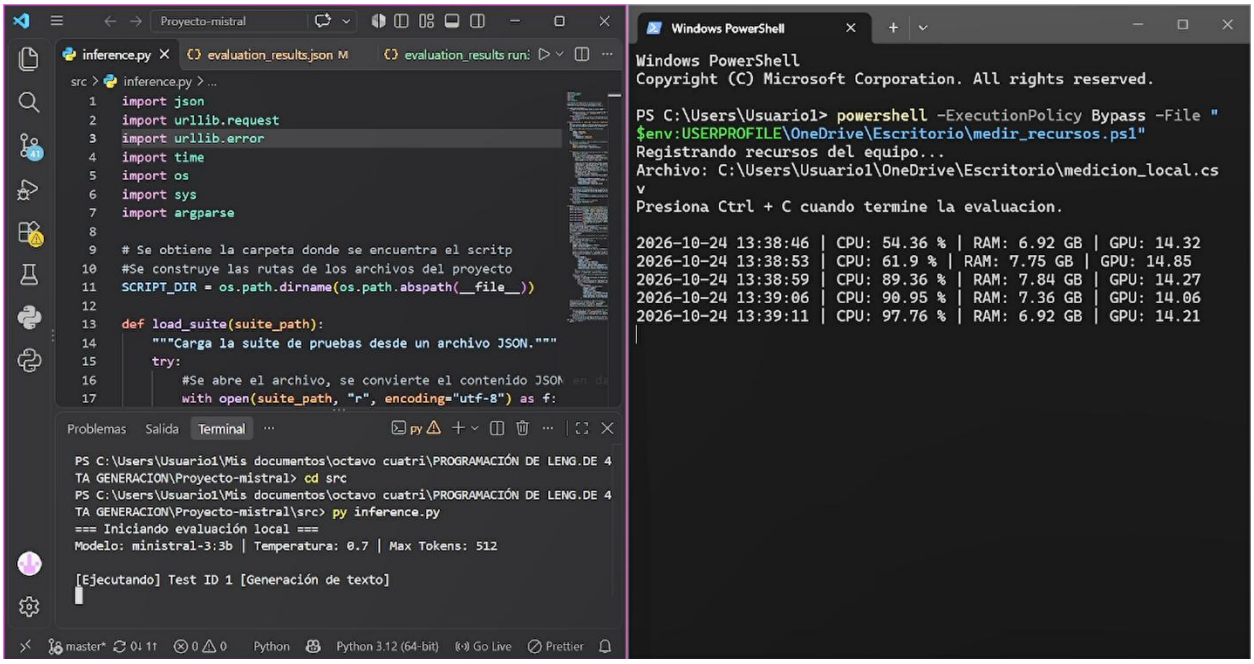
En esta figura, se puede apreciar el resultado de la segunda ejecución, se puede observar que hay resultados diferentes a los que se observan a simple vista en la figura 37.

Figura 39. Resultado de la ejecución Docker run 3



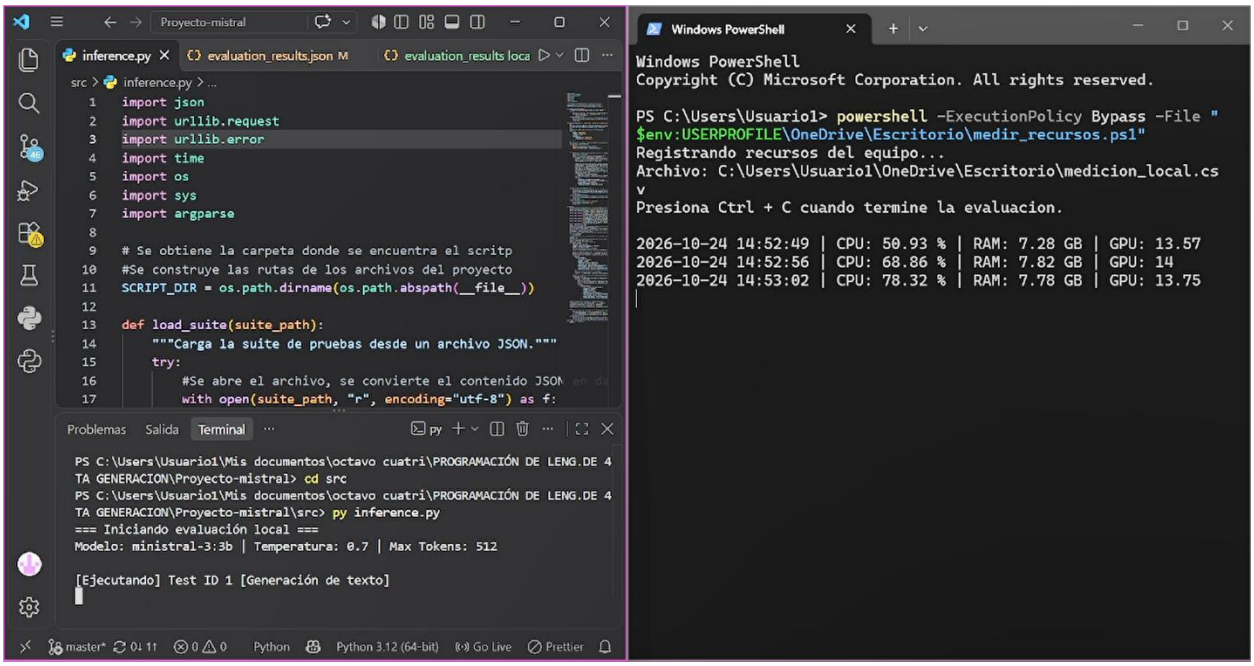
Se presenta la última figura, esta es la última ejecución de Docker run, al igual que las otras dos figuras, se puede apreciar a simple vista las diferencias en los resultados.

Figura 40. Medición local de la ejecución nativa 1



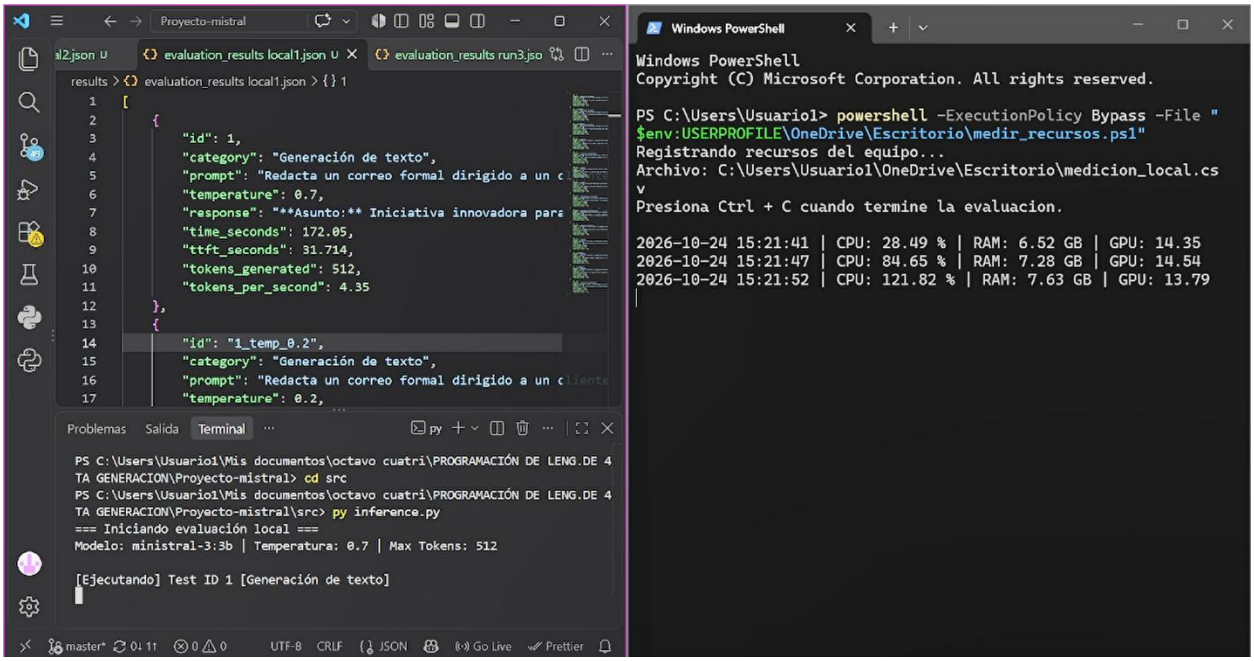
En la figura se puede observar se va dando la medición local de la primera ejecución nativa, se creó un script llamado medir_recursos.ps. Este permite medir el consumo de CPU, RAM y GPU del equipo, por lo que al ejecutar el comando: powershell -ExecutionPolicy Bypass -File "$env:USERPROFILE\OneDrive\Escritorio\medir_recursos.ps1" en powershell como se ve en la figura, se va a recopilar las mediciones cada dos segundos y genera automáticamente el archivo medicion_local.csv que almacenara los datos en un archivo Excel.

Figura 41. Medición local de la ejecución nativa 2



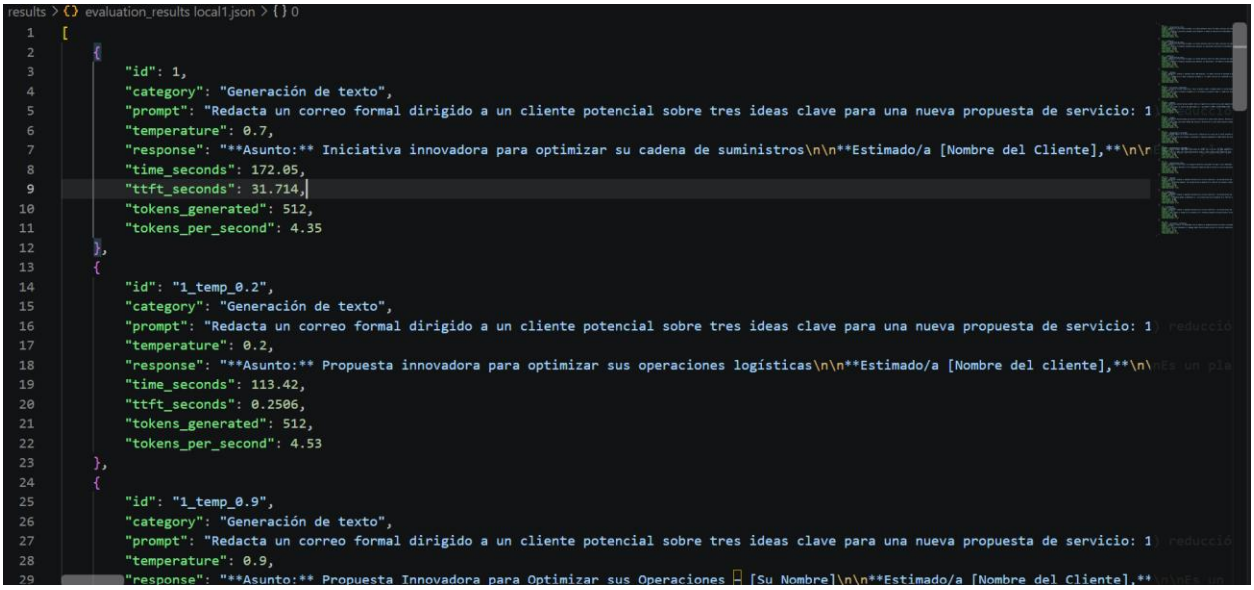
En la figura 41 se observa la segunda ejecución nativa, con el mismo comando recopilando los datos, se puede ver que sus datos son diferentes a los de la figura 40.

Figura 42. Medición local de la ejecución nativa 3



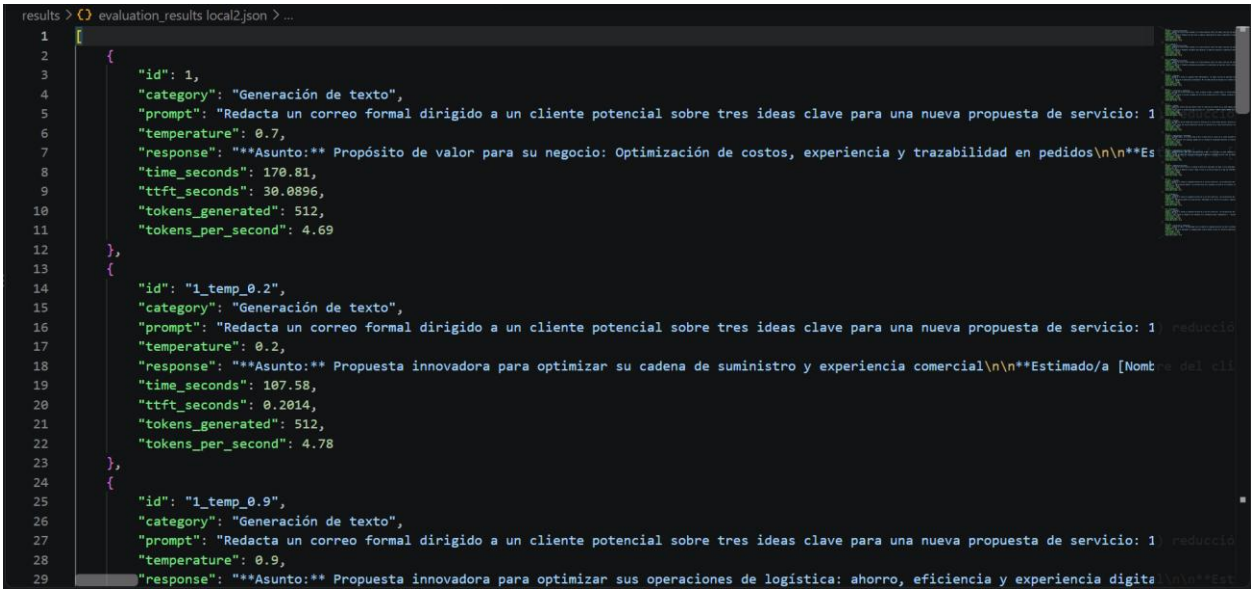
Esta última figura, representa la tercera ejecución nativa, donde se puede ver también la recopilación de sus datos, al igual que las otras figuras, presenta datos diferentes.

Figura 43. Resultado de ejecución nativa 1



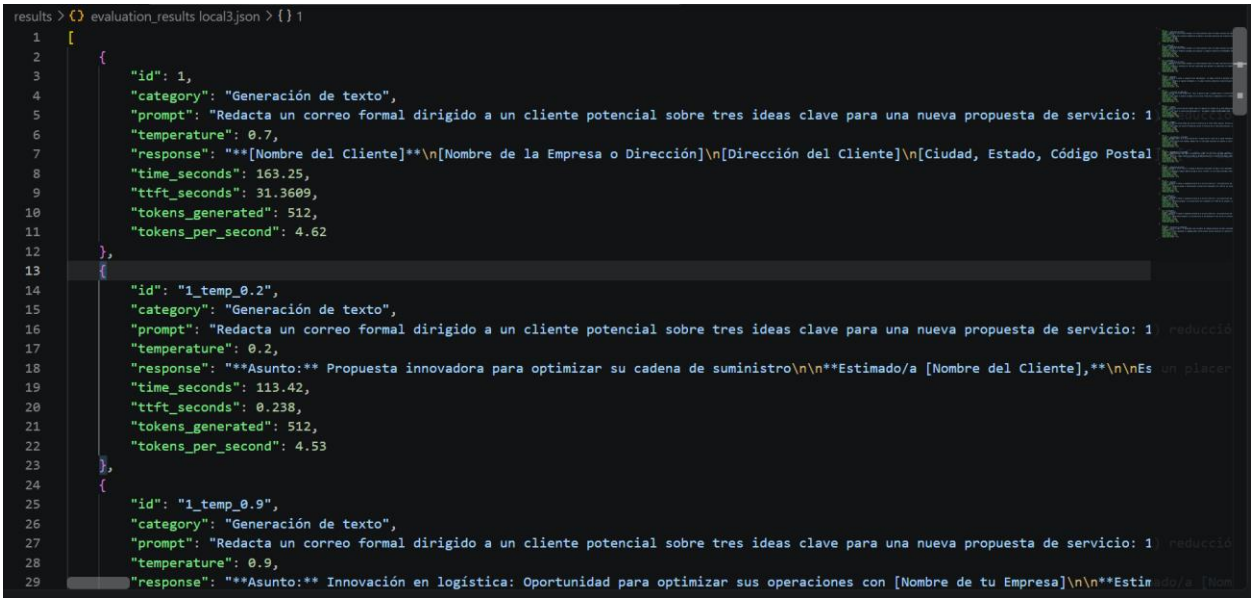
La figura presenta los resultados de la ejecución nativa 1, se puede observar sus time_secondos, TTFT, tokens_generated y tokens_per_second.

Figura 44. Resultado de ejecución nativa 2



Se observa el segundo resultado de la ejecución nativa, se puede ver que tiene diferencias en sus resultados a los que aparecen en la figura 43.

Figura 45. Resultados de ejecución nativa 3



En esta última figura, se puede observar la ejecución nativa 3, que es el ultimo resultado. Que igual a las otras dos, tiene diferencias en sus resultados.

Figura 46. Script para medir recursos

```
$salida = Join-Path ([Environment]::GetFolderPath('Desktop')) 'medicion_local.csv'

"Fecha,CPU_Porcentaje,RAM_Usada_GB,GPU_Actividad_Suma_Porcentajes" |
    Set-Content -Path $salida -Encoding UTF8

Write-Host "Registrando recursos del equipo..."
Write-Host "Archivo: $salida"
Write-Host "Presiona Ctrl + C cuando termine la evaluacion."
Write-Host ""

while ($true) {
    $fecha = Get-Date -Format "yyyy-MM-dd HH:mm:ss"

    # Medir uso total del CPU
    $cpu = $null
    try {
        $contadorCPU = Get-Counter '\Processor Information(_Total)\% Processor Utility' -ErrorAction Stop
        $cpu = [math]::Round($contadorCPU.CounterSamples[0].CookedValue, 2)
    } catch {
        $cpu = $null
    }

    # Medir RAM utilizada
    $memoria = Get-CimInstance Win32_OperatingSystem
    $ramUsadaGB = [math]::Round(
        ($memoria.TotalVisibleMemorySize - $memoria.FreePhysicalMemory) / 1MB, 2
    )

    # Registrar actividad de los motores de la GPU, si esta disponible
    $gpuUso = $null
    try {
        $contadoresGPU = Get-Counter '\GPU Engine(*)\Utilization Percentage' -ErrorAction Stop
        $activos = $contadoresGPU.CounterSamples |
```

La figura 46, muestra el script de medir recursos, el cual se ejecuta en el powershell siendo el comando mencionado en la figura 40, tiene las instrucciones para que mida CPU, RAM y GPU, si se encuentra disponible.

Figura 47. Resultado medición local de ejecución nativa 1

Fecha	CPU_Porcentaje	RAM_Usada_GB	GPU_Actividad_Suma_Porcentajes
2026-10-24 13:38:46	"54,36"	"6,92"	"14,32"
2026-10-24 13:38:53	"61,9"	"7,75"	"14,85"
2026-10-24 13:38:59	"89,36"	"7,84"	"14,27"
2026-10-24 13:39:06	"90,95"	"7,36"	"14,06"
2026-10-24 13:39:11	"97,76"	"6,92"	"14,21"
2026-10-24 13:39:17	"75,01"	"6,99"	"14,41"
2026-10-24 13:39:23	"82,6"	"7,09"	"14,05"
2026-10-24 13:39:28	"73,07"	"7,1"	"14,06"
2026-10-24 13:39:34	"79,59"	"7,14"	"14,48"
2026-10-24 13:39:40	"79,7"	"7,28"	"14,43"
2026-10-24 13:39:45	"90,04"	"7,31"	"14,63"
2026-10-24 13:39:51	"79,83"	"7,34"	"14,33"
2026-10-24 13:39:57	"84,97"	"7,35"	"15,51"
2026-10-24 13:40:03	"85,52"	"7,37"	"15,17"
2026-10-24 13:40:08	"88,81"	"7,39"	"15,07"
2026-10-24 13:40:14	"91,25"	"7,43"	"14,52"
2026-10-24 13:40:21	"93,35"	"7,48"	"15,36"
2026-10-24 13:40:27	"93,62"	"7,53"	"17,2"

En esta figura se observa cómo se almacenan los datos recopilados en el archivo Excel, que fueron necesarios para todo el apartado de comparaciones, así mismo lucen los archivos de las ejecuciones 2 y 3, con sus variaciones en los datos recopilados.

Local frente a nube

Las ejecuciones locales y servicios en la nube, permiten que se utilice modelos de lenguaje, sin embargo, presentan diferencias que debemos destacar, como el rendimiento, sus costos, el mantenimiento y la privacidad, en este apartado se va ir comparando la ejecución local de Ministral 3 3B mediante Ollama que hemos visto durante todo el documento, con dos alternativas que serían la API de Mistral que esta disponible en La Plateforme y una instancia con GPU de Amazon Web

Services por sus siglas en ingles AWS. La comparación de la ejecución local hace uso de las mediciones obtenidas durante las pruebas del proyecto, las alternativas que son en la nube se evalúan a partir de sus características documentadas, así como precios públicos, sin embargo, no se ha ejecutado pruebas prácticas en esos servicios.

Tabla 8. Comparación técnica y práctica

Criterio	**Ollama local**	**API de Mistral**	**Instancia con GPU AWS**
Latencia	El rendimiento de este, depende del hardware, más la configuración, en las pruebas que se estuvieron realizando en el proyecto, el TTFT promedio que se registro fue de 6,586 s.	Esta depende del servicio y de la solicitud, no se mide en este proyecto, así que no hay una cifra comparable.	Va a depender del hardware, así como de la configuración del modelo, tampoco se mide en este proyecto.
Escalabilidad	Es limitada por los pocos recursos disponibles del equipo.	Su capacidad se condiciona por los límites de solicitudes, así como de los tokens de la cuenta.	Las instancias se pueden seleccionar con diferentes recursos de CPU, GPU y memoria.
Mantenimiento	Acá el usuario administra el modelo, así como la instalación local.	La infraestructura de la API es operada por Mistral, el desarrollador tiene la aplicación y credenciales.	En las instancias EC2, las auto administradas, el usuario va a configurar y además mantener el entorno, así como controles de seguridad.
Privacidad	Datos y prompts, procesados localmente no se envían a Ollama.	Acá las solicitudes se procesan en Mistral, su política describe lo que es la retención, contempla excepciones y también tiene opciones según el servicio.	Su protección depende netamente de las configuraciones de la instancia. Así como permisos y controles aplicados.

Internet	No necesita conexión de manera permanente para poder inferir con un modelo ya descargado.	Si requiere el uso de conexión con el servicio de API.	Para el acceso remoto por conexión, se requiere la conectividad, así como de una configuración de red adecuada.
Facilidad de uso	En Ollama se proporciona comando, con una API local para ejecutar modelos.	Se debe integrar por medio de solicitudes a la API, es muy necesario gestionar lo que son las credenciales, así como límites de uso.	Se ofrece más control en el entorno, sin embargo, implica que se configure y administre la instancia.

Según Amazon Web Services (s.f), las instancias EC2 G5, están diseñadas para cargas de trabajo gráficas, así de aprendizaje automático, y su modelo g5.xlarge dispone de 4vCPU, 16 GiB de memoria y GPU NVIDIA A10G con 24 GB de memoria GPU, dichas características permiten disponer de más recursos de GPU, a diferencia de una computadora que no cuenta con tarjeta gráfica dedicada. Sin embargo, todo esto, no garantiza por sí solo una latencia específica, ya que el rendimiento depende del modelo y configuración.

Del lado económico, de API de Mistral, en su página oficial de precios, se indica una tarifa de $0,10 dólares, por millón de tokens de entrada, con $0,10 dólares, por millón de tokens de salida para Mistral 3 3B (Mistral AI, s.f). Así que, para crear el cálculo, se crea un escenario de 1000 consultas diarias durante 30 días, lo que equivale a 30.000 consultas mensuales, se asume que cada consulta utiliza 250 tokens de entrada, donde genera 250 tokens de salida, donde ambos forman 500 tokens, todo esto solo es un supuesto cálculo, no un resultado medido.

- Consulta al mes: 1000 x 30 = 30.000
- Tokens de entrada: 30.000 x 250 = 7.500,000 tokens

- Tokens de salida: 30.000 x 250 = 7.500,00 tokens
- Costo de entrada: Un costo de 7,5 millones x $0,10 dólares = $0,75 dólares
- Costo de salida: un costo de 7,5 millones x $0,10 dólares = $0,75 dólares

Con todo esto el costo mensual que se estima de la API seria de unos $1,50 dólares, este cálculo contempla lo que sería el consumo de tokens de Ministral 3 3B, con las tarifas que se consultaron, pero, no incluye el impuesto, ni tampoco costos adicionales por algún otro servicio. La tarifa debe verificarse nuevamente para contratar el servicio, puesto que los precios pueden cambiar.

Por otro lado, tenemos la instancia con GPU en AWS, como alternativa se considera una instancia de EC2 g5.xlarge, los precios en Economize actualizada el 8 de octubre de este año, logra mostrar un costo bajo demanda de unos $1,006 dólares por hora, esto equivale aproximadamente a unos $734,38 dólares al mes, si llega a permanecer encendida durante unos 730 horas (Economize, 2026).

- Costo por hora: $1,006
- Horas estimadas por mes: 730h
- Costo mensual: Unos $1,006 dólares x 730 h = $734,38 dólares aproximados.

La estimación anterior corresponde si es que se mantiene la instancia de manera que funcione durante todo el mes, esto no incluye necesariamente almacenamiento adicional, ni transferencia de datos, como de impuestos o servicios. Si por alguna razón la instancia se detiene cuando no se está utilizando, los costos de cómputo pueden reducirse, pero, otros recursos asociados si pueden seguir generando cargos.

Por otro lado, tenemos la ejecución local con Ollama, en Ollama, no se paga una tarifa por cada token generado, pero, sí está la existencia de costos indirectos, que estos serían el consumo eléctrico, el mantenimiento del equipo, así como posibles actualizaciones de hardware, sin embargo, no se midió el consumo eléctrico del equipo donde se esta trabajando, ni tampoco se dispone de una tarifa eléctrica aplicada a las pruebas, por lo que, no se asignó un costo mensual numérico en esta parte.

Tabla 9. Comparación de costos

Alternativa	**Costo mensual (solamente estimado)**	**Consideración**
Ollama local	No se calculo	El consumo eléctrico y de mantenimiento del equipo.
API de Mistral	$1,50 dólares	Consultas al mes estimadas 30.000, con 250 tokens para entrada y salida de consulta.
AWS g5.xlarge	$734,38 dólares	La instancia funcionando durante 730 horas, y puede existir cargos adicionales.

Los valores que se dan, no representan una comparación que sea idéntica de infraestructura, puesto que la API cobra según los tokens utilizados, mientras que, con las instancias de AWS, se cobra por el tiempo de uso de recursos aprovisionados, por esto, la API tiene un costo de entrada menor en el escenario que se planteó, mientras que, por otro lado, AWS ofrece la maquina con GPU que permite al usuario configurar y administrar.

Como parte final de este apartado, las recomendaciones serían que:

- Ejecución local: permite mayor control sobre los datos, se puede trabajar sin internet, evita pagos por API, pero, la capacidad depende bastante del equipo, como se vio a lo largo de este proyecto por las limitaciones del equipo.

- API de Mistral: tiene una facilidad para poder integrar modelos, sin la necesidad de administrar servidores, ni comprar hardware especializado. Pero, requiere internet, además, se debe considerar que implica costos por consumo y exige revisar las condiciones de privacidad.
- GPU en AWS: ofrece los recursos configurables, tiene mayor capacidad de cómputo, pero, va a requerir de conocimiento técnico, además, de un control cuidadoso de los costos.

En conclusión, la elección que se haga va a depender de las necesidades, si es una ejecución local para autonomía, si con API por facilidad de uso y su infraestructura o la GPU en la nube por la mayor capacidad de cómputo, las recomendaciones se basan únicamente en lo documentado anteriormente y cálculos económicos, pero no con pruebas de latencia en la nube.

Documentación técnica

Los problemas encontrados durante este proyecto, fueron los siguientes:

Tabla 10. Problemas encontrados

Problema	**Causa**	**Solución aplicada**	**Fuente que se consulto**
Inicialmente se ejecutó el ministral incorrecto, se había ejecutado el mistral:latest, ya se había comenzado ciertas partes del proyecto con dicho modelo, originalmente se tenia que ejecutar el ministral-3:3b.	Se había ejecutado el comando incorrecto del modelo en la terminal, se había ejecutado el comando: run mistral:latest, el cual no funcionaba bien en las pruebas, porque forzaba más al equipo.	Después de investigar en la página de Ollama, se cambió el modelo a ministral-3:3b, el cual cumplía por completo con las características del equipo. Se corrigió el código para que las ejecuciones fueran con el modelo correcto, por lo que se volvió hacer todas las pruebas.	Se consulto en la página oficial de Ollama.com, buscando en modelos el Ministral-3:3b.

Conclusiones

Este proyecto realizado, ha permitido relacionar fundamentos teóricos de la IA, con la parte de la implementación y la evaluación de Ministral 3 3B, el proyecto permite comprender lo que son capacidades, así como limitaciones de los modelos de lenguaje para el procedimiento y la generación de información.

La selección de Ministral 3 3B, con una cuantización de Q4_K_M, fue una opción excelente por las características del equipo con el que se realizó este proyecto, puesto que tiene limitaciones como 8 GB de RAM, con gráficos integrados, y no contar con una GPU dedicada. Las pruebas que se realizaron pudieron demostrar que la ejecución depende de los recursos disponibles, que la cuantización, aunque facilite el uso del modelo, no elimina lo que es las limitaciones de rendimiento.

Por otro lado, las comparaciones realizadas a Ollama entre Docker, evidenció las diferencias de rendimiento que tienen estas dos. Las pruebas funcionales, permitieron que se pudiera observar la importancia de evaluar las respuestas de cada tarea, ya que los resultados presentan limitaciones claras.

Por otra parte, la parte ética, la ejecución local nos ofrece mayor control sobre los datos, sin embargo, no va a garantizar la privacidad, ni tampoco exactitud de las respuestas, por esto es necesario proteger la información y hacer validaciones de los resultados.

Para finalizar este apartado de documentación, el prompting puede considerarse como una posible evolución de los lenguajes de cuarta generación por sus siglas 4GL, ya que, permite poder expresar lo que se desea mediante lenguaje natural, aun así, los modelos llegan a tener interpretaciones incorrectas de las instrucciones o bien, generar respuestas erróneas, que no son

coherentes, por esto el uso requiere de una evaluación crítica, asimismo la validación de los resultados.

Referencias

Stryker, C., & Kavlakoglu, E. (2026, 2 de octubre). *¿Qué es la inteligencia artificial (IA)?* IBM. https://www.ibm.com/mx-es/think/topics/artificial-intelligence

Mucci, T. (2024, 21 de octubre). *La historia de IA*. IBM. https://www.ibm.com/mx-es/think/topics/history-of-artificial-intelligence

Sagula, J. (2024, 01 de abril). *La Inteligencia Artificial, un flujo continuo desde su nacimiento hasta ahora*. Departamento Ciencias Básicas-Universidad Nacional de Luján, Argentina. https://journalsibemir.com/index.php/jsibemir/article/view/3/4

Faggella, D. (2016, 11 de octubre) *¿Qué es la Inteligencia Artificial? Una definición fundamentada*. Emerj. https://emerj.com/what-is-artificial-intelligence-an-informed-definition/

Moreno, J. (2024, 09 de enero). *Una panorámica de los sistemas de inteligencia artificial desde la perspectiva del derecho administrativo*. Universidad de Zaragoza. https://revistaaragonesa.aragon.es/index.php/raap/article/view/120

Boesch, G. (2026, 10 de junio). *Los tipos de inteligencia artificial: ANI, AGI y ASI*. Viso.ai. https://viso.ai/deep-learning/artificial-intelligence-types/

Stryker, C. (2021, 06 de octubre) *¿Qué son los LLM (grandes modelos de lenguaje)?* IBM. https://www.ibm.com/mx-es/think/topics/large-language-models

Bergmann, D. (2025, 15 de septiembre) *¿Qué es el aprendizaje profundo?* IBM. https://www.ibm.com/mx-es/think/topics/deep-learning

Bergmann, D. (2026, 2 de octubre) *¿Qué es el aprendizaje automático?* IBM. https://www.ibm.com/mx-es/think/topics/machine-learning

Risco, M. (2025, 11 de noviembre) *¿Cuál es la diferencia entre inteligencia artificial y machine learning? Guía completa para entender*. IAgrup. https://www.iagrup.com/blog/cual-es-la-diferencia-entre-inteligencia-artificial-y-machine-learning/

Gómez, E. (2022, 08 de noviembre). *Diferencia entre machine learning y deep learning.* Universitat Oberta de Catalunya. https://blogs.uoc.edu/informatica/es/machine-learning-vs-deep-learning-diferencias/

Nanda, A. (2024, 03 de septiembre). *Modelos Generativos vs Discriminativos: Diferencias y casos de uso*. Datacamp. https://www.datacamp.com/es/blog/generative-vs-discriminative-models

Bergmann, D., & Stryker. C. (2024, 05 de diciembre) *¿Qué es un mecanismo de atención?* IBM. https://www.ibm.com/mx-es/think/topics/attention-mechanism

Vaswani, A. et al. (2017, 12 de junio). *Attention Is All You Need*. Arxiv. https://arxiv.org/abs/1706.03762

Chourey, L. (2024, 16 de diciembre). *Ciclo de vida de los modelos de lenguaje a gran escala: un examen exhaustivo de los desafíos de la capacitación y la implementación*. Computer society. https://www.computer.org/publications/tech-news/trends/large-language-model-lifecycle

Zoumana, K. (2024, 11 de diciembre). *Llama.cpp Tutorial: Una Guía Completa para la Inferencia e Implementación Eficiente del LLM*. Datacamp. https://www.datacamp.com/es/tutorial/llama-cpp-tutorial

Shekhar, A. (2026, 16 de junio) *¿Cómo funciona vLLM?* Escuela de resultados. https://outcomeschool.com/blog/how-does-vllm-work

Bergmann, D., & Stryker, C. (2023, 4 de octubre) *¿Qué es PyTorch?* IBM. https://www.ibm.com/mx-es/think/topics/pytorch

Bittencourt, J. (2023, 28 de mayo). *Framework: ¿qué es y para qué sirve esta herramienta?* Aluralatam. https://www.aluracursos.com/blog/framework-que-es-y-para-que-sirve

Bigelow, S. (2024, 16 de julio). *Unidad de procesamiento tensorial (TPU)*. Tech target. https://www.techtarget.com/whatis/definition/tensor-processing-unit-TPU

Hernández, M. (2026, septiembre). *Análisis e Interpretación de Arquitecturas MoE para Clasificación mediante Patrones de Selección de Experto*. Facultade de Informática Universidade da Coruña. https://ruc.udc.es/entities/publication/ecc7cdb6-1e8b-4a28-ba03-f7f1343f6b06

Yugsi, M. (2026, 23 de febrero). *Desarrollo de la interfaz de validación de 3 modelos LLM*. Universidad de las Fuerzas Armadas. https://repositorio.espe.edu.ec/bitstreams/61959cb5-5261-4e58-8f74-9ee2697dff82/download

Yana. (2024, 29 de noviembre). *GPU*. Qsfptek. https://www.qsfptek.com/es/network-glossary/gpu.html?srsltid=AU7gw4V-fgpIIZSW_39veDZFgS3Z_H_a3Hiy2R7skwOR8ejZ65OGQcL3

Gómez, B. (2021, 01 de enero). *VRAM: qué es y para qué sirve en las tarjetas gráficas*. Profesional review. https://www.profesionalreview.com/2021/01/01/vram-que-es/

Aller, A. (2024, 04 de junio) *¿Qué es la VRAM? Descubre porqué es tan importante en gaming*. Mundo Pccomponentes. https://www.pccomponentes.com/blog/vram-memoria-de-video?srsltid=AU7gw4UljVa0T5KUpyhA7V-_0GTIV_QBubE9f2vSObl6b0m6WfDGfsOa

Bergmann, D., & Stryker, C. (2025, 28 de marzo) *¿Qué es un modelo de transformador?* IBM. https://www.ibm.com/mx-es/think/topics/transformer-model

Filipan, M. (2025, 08 de septiembre). *Inferencia frente a entrenamiento: por qué ejecutar IA es distinto de crearla*. Dweve. https://dweve.com/es/blog/inference-vs-training-explained/

Mitchell, M., & Krakauer, D. (2023, 21 de marzo). *El debate sobre la compresión en los grandes modelos de lenguaje de la IA*. PNAS. https://www.pnas.org/doi/10.1073/pnas.2215907120

Bellón, F. (2026, 16 de septiembre). *Hassabis, el gurú bueno de la IA: Estamos a menos de tres años de la inteligencia artificial general*. Nueva Revista. https://www.nuevarevista.net/demis-hassabis-ia-singularidad/

Bergmann, D., & Downie, A. (2026, 14 de julio) *¿Qué es la inteligencia artificial general (IAG)?* IBM. https://www.ibm.com/think/topics/artificial-general-intelligence

Werner, J. (2026, 30 de enero). *Yann LeCun sobre la inteligencia artificial general y los bienes comunes digitales*. Forbes. https://www.forbes.com/sites/johnwerner/2026/01/27/yann-lecun-on-artificial-general-intelligence-and-the-digital-commons/

Sonny, A. (2023, 29 de octubre). *AGI: avances y desafíos de la inteligencia artificial general*. Noticias AI. https://noticias.ai/avances-y-desafios-en-el-desarrollo-de-la-inteligencia-artificial-general-agi/

Tejada, J. (2026, 26 de febrero). *El desafío de la conciencia en la inteligencia artificial*. Estudios de la ciencia. https://estudiosciencia.com/2026/02/26/el-desafio-de-la-conciencia-en-la-inteligencia-artificial/

Ayago, I. (2025, 17 de marzo) *¿Qué es la Inteligencia Artificial General (AGI)?* Software Trading IA. https://whaleanalytics.tech/la-inteligencia-artificial-general-agi-y-sus-riesgos-estamos-preparados/

Medha, M. (2026, 08 de enero). *Sesgo de IA: 16 ejemplos reales de sesgo de IA y guía de mitigación*. Crescendo. https://www.crescendo.ai/blog/ai-bias-examples-mitigation-guide

Barredo, E. (2024, 30 de mayo). *El caso Scarlett Johansson: una alerta sobre el uso de datos personales por la IA*. Universidad Isabel I. https://www.ui1.es/blog-ui1/el-caso-scarlett-johansson-una-alerta-sobre-el-uso-de-datos-personales-por-la-ia

Gomstyn, A., & Jonker, A. (2025, 08 de enero). *Desinformación de la IA: cómo reducir la exposición y el riesgo de su empresa*. IBM. https://www.ibm.com/mx-es/think/insights/ai-misinformation

Microsoft. (2026, 28 de septiembre). *Inferencia de IA en Windows Server*. Microsoft Ignite. https://learn.microsoft.com/en-us/windows-server/ai/local-ai-inference

Jaf, S., Barakat, B., Cheng, Y. (2026, 13 de enero). *IA sin censura en la práctica: seguimiento de modelos de aprendizaje automático disponible públicamente y de implementación local*. MDPI. https://www.mdpi.com/1999-5903/17/10/477

Asamblea Legislativa. (2026, 07 de abril). *Consulta sobre proyecto de ley tramitado bajo expediente 24875*. Asamblea Legislativa de la República de Costa Rica. https://www.asamblea.go.cr

Ministerio de Ciencia, Innovación, Tecnología y Telecomunicaciones. (2024, 24 de octubre). *MICITT presentó Estrategia Nacional de Inteligencia Artificial (ENIA).* Ministerio de Ciencia,

Innovación, Tecnología y Telecomunicaciones. https://micitt.go.cr/el-sector-informa/micitt-presento-estrategia-nacional-de-inteligencia-artificial-enia

Asamblea Legislativa de la República de Costa Rica, Comisión Permanente Especial de Derechos Humanos. (2025, 06 de marzo). *Expediente 23.919 Ley para la promoción responsable de la inteligencia artificial en Costa Rica*. Asamblea Legislativa de la República de Costa Rica, Comisión Permanente Especial de Derechos Humanos. https://delfino.cr/asamblea/proyecto/23919

Dirección General de Comunicación. (2024, 01 de agosto). *El Reglamento de inteligencia Artificial entra en vigor*. Web oficial de la UE. https://commission.europa.eu/news-and-media/news/ai-act-enters-force-2024-08-01_es

Mistral. (2025, 02 de diciembre). *Presentamos Mistral 3*. Mistral. https://mistral.ai/news/mistral-3/

Mistral. (s.f). *Ministral 3 3B Instrucción 2512 BF16*. Huggingface. https://huggingface.co/mistralai/Ministral-3-3B-Instruct-2512-BF16

Mistral. (2025, 02 de diciembre). *Ministral 3 3B*. Mistral. https://docs.mistral.ai/models/ministral-3-3b-25-12

Kamath, A. et al. (2025, 12 de marzo). *Informe técnico de Gemma 3*. Arxiv. https://arxiv.org/abs/2503.19786

Yang, A. et al. (2025, 15 de mayo). *Informe técnico de Qwen3*. Arvix. https://arxiv.org/abs/2505.09388

Google AI. (2026, 01 de abril). *Condiciones de uso de Gemma*. Google AI. https://ai.google.dev/gemma/terms

Qwen. (s.f). *Lo más destacado de Qwen3*. Huggingface. https://huggingface.co/Qwen/Qwen3-4B

Economize. (2026, 25 de octubre). *g5.xlarge*. economize. https://www.economize.cloud/resources/aws/pricing/ec2/g5.xlarge/

Mistral IA. (s.f). *Precios de API*. Mistral IA. https://docs.mistral.ai/inference/pricing

Amazon Web Services. (s.f). *Instancias G5 de Amazon EC2*. Amazon Web Services. https://aws.amazon.com/es/ec2/instance-types/g5/

Jberg5. (s.f). *Utilice la API de Ollama para crear aplicaciones con modelos en la nube o locales*. Github. https://github.com/ollama/ollama/blob/main/docs/api/introduction.mdx

Mistral IA. (2026, 03 de septiembre). *Política de privacidad*. Mistral IA. https://legal.mistral.ai/terms/privacy-policy/?lang=en

Amazon Web Services. (s.f). *Requisitos previos para EC2 Instance Connect*. Amazon Web Services. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-connect-prerequisites.html

Rahn, A. (s.f) *¿Cómo puedo actualizar Ollama?* Github. https://github.com/ollama/ollama/blob/main/docs/faq.mdx

Mistral IA. (s.f). *Uso y límites*. Ministral IA. https://docs.mistral.ai/admin/billing-usage/usage-limits

Anexo

Durante el desarrollo del proyecto se utilizaron algunas herramientas de inteligencia artificial como apoyo:

Tabla 11. Uso de herramientas de IA

Herramienta	**Utilización**	**Secciones donde apoyo**
Ministral 3 3B	Se hizo usó para generar el código de inference.py, al igual que generar el prompts que se usó para las pruebas.	Sección 2 suite de pruebas funcionales y sección 3 script propio de inferencia en Python.
Gemini	Su usó fue para poder identificar los errores que se presentaban en el código que creo Ministral 3 3B.	Sección 3 script propio de inferencia en Python.
ChatGPT	Se utilizo más que todo para la orientación de comandos en la terminal, para poder ejecutar el modelo de Ministral correcto, y los comandos correctos para poder ejecutar las pruebas.	Sección 2 suite de pruebas funcionales, la parte de ejecuciones en el terminal así mismo aplicado para la sección 4 de contenerización con Docker.